

Mizar Evo

Detailed discussion deck with presenter notes for the Białystok Mizar team

Mizar Evo project

September 2026

1.1 - Three Pillars (1/2)

The design is judged by three constraints that must hold at the same time.

Pillar	What it means	Why it matters
Readability	Proof scripts stay close to mathematical exposition; local disambiguation is preferred when ambiguity matters.	Human reviewers can inspect the argument without replaying hidden automation.
AI-readiness	Verifier context, source spans, obligations, and safe edit classes are exposed through small auditable interfaces.	AI can help with search, explanation, migration, and patch proposals without becoming trusted proof authority.
Scalability	Imports, artifacts, packages, caches, and publication links have stable boundaries.	Large-library maintenance becomes predictable without changing theorem meaning.

The design is judged by three constraints that must hold at the same time.

Why?	How it works	Why it matters
Flexibility	Proof sets may draw on mathematical expressions; local shared queries to persistent object graphs; macros.	How an extension can inspect the program: makes replacing hidden assertions.
Modularity	Smaller context, more space, subgraphs, and sets of domains exposed through model.	It can talk with math, expressions, migration, and patch dependencies; becoming proved proof models.
Scalability	In parts, artifacts, packages, patches, and packages that have made boundaries.	Long history of maintenance across production without changing theorem meaning.

Presenter script: Read aloud:

- The design is judged by three constraints that must hold at the same time.

For the table, read the row labels first, then the design reason.

The three pillars reinforce each other:

Readable source

- > better human review
- > better AI retrieval and edits
- > clearer artifacts and dependency traces
- > more reliable large-library maintenance

Design test:

- A feature is good only if it improves automation or scale while preserving readable mathematics.

Readable source
→ better human review
→ better AS verification and edits
→ cleaner artifacts and dependency traces
→ more reliable large-library maintenance

- A feature is good only if it improves automation or scale while preserving readable mathematics.

Presenter script: Read aloud:

- A feature is good only if it improves automation or scale while preserving readable mathematics.

After the example, say which design obligation the syntax makes visible.

Key Phrase:

Mizar Evo should not make proofs shorter by making the argument invisible.

Presenter script: Say:

- This is the key contrast with very tactic-heavy workflows.
- Some automation is valuable, but the accepted proof artifact must remain inspectable.

Key Points:

- A first-order set-theoretic core matches much ordinary mathematical practice.
- ATP tools are strong in first-order reasoning.
- Soft types, modes, attributes, and registrations can remain Mizar-facing structure around that core.
- The kernel can focus on checking evidence rather than performing high-level elaboration.

Design Reason:

- This choice preserves the current Mizar foundation while letting powerful ATPs work at the layer where they are strongest.
- It also avoids making dependent-type elaboration or tactic execution part of the trusted explanation of ordinary mathematical text.

Key Points:

- A first-order set-theoretic core matches much ordinary mathematical practice.
- ATP tools are strong in first-order reasoning.
- Soft types, modes, attributes, and registrations can remain Mizar-facing structure around that core.
- The kernel can focus on checking evidence rather than performing high-level elaboration.

Design Reason:

- This choice preserves the current Mizar foundation while letting powerful ATPs work at the layer where they are strongest.
- It also avoids missing dependent-type elaboration or tactic execution part of the current explanation of ordinary mathematical text.

Presenter script: Read aloud:

- A first-order set-theoretic core matches much ordinary mathematical practice.
- ATP tools are strong in first-order reasoning.
- Soft types, modes, attributes, and registrations can remain Mizar-facing structure around that core.
- The kernel can focus on checking evidence rather than performing high-level elaboration.
- This choice preserves the current Mizar foundation while letting powerful ATPs work at the layer where they are strongest.

Key Points:

- explicit module imports;
- templates for parameterized definitions and theorem schemas;
- package manifests and lockfiles;
- stable fully-qualified names;
- deterministic build artifacts;
- incremental and parallel verification;
- LSP for editors;

Key Points:

- planned MCP-style context surfaces for AI agents;
- versioned links from publications to library objects.

Key Points:

- explicit module imports;
- templates for parameterized definitions and theorem schemas;
- package manifests and lockfiles;
- stable fully-qualified names;
- deterministic build artifacts;
- incremental and parallel verification;
- LSP for editors;

Key Points:

- shared MCP-style content interfaces for AI agents;
- versioned links from `path` to library objects.

Presenter script: Read aloud:

- explicit module imports;
- templates for parameterized definitions and theorem schemas;
- package manifests and lockfiles;
- stable fully-qualified names;
- deterministic build artifacts;

Key Points:

- not abandoning declarative proof style;
- not making ATP success trusted by default;
- not replacing human mathematical review with AI;
- not forcing every old article into a new style at once.

Key Points:

- not abandoning declarative proof style;
- not making ATP success trusted by default;
- not replacing human mathematical review with AI;
- not forcing every old article into a new style at once.

Presenter script: Read aloud:

- not abandoning declarative proof style;
- not making ATP success trusted by default;
- not replacing human mathematical review with AI;
- not forcing every old article into a new style at once.

Key Points:

- Preserve the recognizable mathematical surface.
- Make dependencies and names more explicit.
- Separate structure definition, inheritance mapping, and template parameterization.
- Give tools stable identities and source spans.
- Keep compatibility metadata for migrated MML material.

Key Points:

- Preserve the recognizable mathematical surface.
- Make dependencies and names more explicit.
- Separate structure definition, inheritance mapping, and template parameterization.
- Give tools stable identities and source spans.
- Keep compatibility metadata for migrated MML material.

Presenter script: Read aloud:

- Preserve the recognizable mathematical surface.
- Make dependencies and names more explicit.
- Separate structure definition, inheritance mapping, and template parameterization.
- Give tools stable identities and source spans.
- Keep compatibility metadata for migrated MML material.

2.2 - Mizar To Evo Language Delta Inventory (1/7)

This is a checklist, not a claim that every item must be finalized before the visit. Artifact and workflow deltas that affect migration semantics follow in the next inventory slide.

Area	Current Mizar baseline	Evo delta	Why it changes
Article environment	<code>environ</code> roles such as vocabularies, notations, constructors, registrations, requirements	<code>explicit import</code> prelude and module/package namespace	make the dependency surface reproducible and reviewable
Module interfaces	article export is mediated by library tooling, with fewer source-level interface boundaries	<code>export</code> , <code>public/private</code> , opaque imports, and separate compilation define the public API surface	keep reusable interfaces stable while proof bodies and helpers stay private
Symbol identity	article-local names and MML article labels	path-derived FQNs and module-qualified labels	support migration, renaming, API compatibility, and citation repair

Part 2. Language Specification

2.2 - Mizar To Evo Language Delta Inventory (1/7)

This is a checklist, not a claim that every item must be finalized before the visit. A staff

[illegible]

Presenter script: Say:

- The important review question is not "how many syntactic changes exist?" but "which Mizar-facing idea is being preserved, and which hidden dependency is being made explicit?"

Read aloud:

- This is a checklist, not a claim that every item must be finalized before the visit. Artifact and workflow deltas that affect migration semantics follow in the next inventory slide.

For the table, read the row labels first, then the design reason.

2.2 - Mizar To Evo Language Delta Inventory (2/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Symbolic notation and aliases	broad symbolic constructor vocabulary, with synonyms and antonyms as mathematical aliases	arbitrary symbols concentrated in <code>func</code> and <code>pred</code> ; type constructors stay identifier-like; synonym/antonym equivalence is preserved	stabilize lexing, parsing, editor tooling, and AI edits without losing alternative notation
Operator precedence and parse domains	notation is powerful but precedence choices can be implicit in parser/verifier behavior	explicit operator metadata, separate term/formula precedence domains, and parse-before-overload discipline	make grouping deterministic, reviewable, and independent of overload choice
Lexical activation	vocabulary and notation availability is article-environment driven	import-prelude pre-scan, source-position active lexicon, no forward references, operator metadata after declaration	make tokenization and dot/operator disambiguation deterministic

Mizar Evo

└ Part 2. Language Specification

└ 2.2 - Mizar To Evo Language Delta Inventory (2/7)

2.2 - Mizar To Evo Language Delta Inventory (2/7)

Item	Current Mizar Feature	New Delta	Use in Mizar
Symbolic variables and lambda	Symbolic variables and lambda, with lambda abstraction in mathematical logic	Symbolic variables abstraction in lambda abstraction in lambda abstraction in lambda	Symbolic variables, lambda, lambda (lambda, lambda of lambda) lambda (lambda, lambda of lambda) lambda
Operational predicates and quantifiers	Operational predicates and quantifiers, with lambda abstraction in mathematical logic	Operational predicates abstraction in lambda abstraction in lambda abstraction in lambda	Operational predicates, lambda, lambda (lambda, lambda of lambda), lambda (lambda, lambda of lambda), lambda
Logical predicates	Logical predicates and lambda, with lambda abstraction in mathematical logic	Logical predicates abstraction in lambda abstraction in lambda abstraction in lambda	Logical predicates and lambda (lambda, lambda of lambda), lambda (lambda, lambda of lambda), lambda

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Mizar To Evo Language Delta Inventory (2/7). Point to the visible example, question, or diagram before moving on.

2.2 - Mizar To Evo Language Delta Inventory (3/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Soft type foundation	Mizar soft types over set-theoretic objects	preserve soft typing while making radix/mode heads, object/set, type erasure, widening/narrowing, and reconsider obligations explicit	keep Mizar's foundation recognizable while exposing verifier obligations
Structures	parent structure, fields, and selectors are tightly coupled in one declaration	struct, field, property, and inherit are separate concepts	distinguish layout, derived canonical values, and inherited views
Inheritance	inherited fields are mostly implicit in structure syntax	inherit Child extends Parent where ... records mapping, renaming, narrowing, and coherence	make multiple inheritance and diamond cases auditable

Part 2. Language Specification

2.2 - Mizar To Evo Language Delta Inventory (3/7)

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Mizar To Evo Language Delta Inventory (3/7). Point to the visible example, question, or diagram before moving on.

Area	General MLG	Core MLG	Other MLG
Polysyllable formation	Minimal syllable must be a syllable	permissible syllable must be a syllable	permissible syllable must be a syllable
Stress	permissible syllable must be a syllable	permissible syllable must be a syllable	permissible syllable must be a syllable
Intonation	permissible syllable must be a syllable	permissible syllable must be a syllable	permissible syllable must be a syllable

2.2 - Mizar To Evo Language Delta Inventory (4/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Modes and attributes	soft type vocabulary with adjectives and clusters	modes remain type abbreviations/refinements; attributes are type-refining predicates	preserve Mizar style while separating classification from data layout
sethood and comprehensions	Fraenkel-style set formation relies on set-theoretic side conditions	mode sethood and comprehension checks are explicit type-checker obligations	preserve set-theoretic foundations before ATP search starts
Definitions and correctness	func/pred definitions use familiar equals/means styles	preserve equals, means, existence, uniqueness, and assume side conditions as explicit obligations	keep Mizar's definitional idiom while making obligations auditable

Mizar Evo

└ Part 2. Language Specification

└ 2.2 - Mizar To Evo Language Delta Inventory (4/7)

Area	Current Mizar Feature	How Delta	Why Delta
Order and notation	Order is not specified in the language; it is implied by the order of the arguments in the function call.	Order is not specified in the language; it is implied by the order of the arguments in the function call.	Order is not specified in the language; it is implied by the order of the arguments in the function call.
Notation and composition	Notation is not specified in the language; it is implied by the order of the arguments in the function call.	Notation is not specified in the language; it is implied by the order of the arguments in the function call.	Notation is not specified in the language; it is implied by the order of the arguments in the function call.
Definition and construction	Definition is not specified in the language; it is implied by the order of the arguments in the function call.	Definition is not specified in the language; it is implied by the order of the arguments in the function call.	Definition is not specified in the language; it is implied by the order of the arguments in the function call.

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Mizar To Evo Language Delta Inventory (4/7). Point to the visible example, question, or diagram before moving on.

2.2 - Mizar To Evo Language Delta Inventory (5/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Predicate and functor properties	algebraic properties support concise automatic reasoning	proof-backed property declarations such as commutativity, symmetry, and irreflexivity	keep compact mathematical notation while recording exactly what automation may use
Overload and redefine	overloaded symbols and redefinitions are central Mizar idioms	root/family overload resolution, coherence with, and refinement joins are recorded and explainable	avoid source-order ambiguity and make refined result facts stable
Registrations, clusters, and reductions	powerful automatic propagation and simplification, often hard to explain locally	labeled registration items, import-filtered resolution graph, reduction index, trace artifacts	keep automation but make inference and rewriting explainable

Mizar Evo

└ Part 2. Language Specification

└ 2.2 - Mizar To Evo Language Delta Inventory (5/7)

Item	Current Mizar Delta	See Delta	Why, I thought
Prologue and locale properties	Domain properties (upper) locale must not be empty	Domain must not be empty, locale must not be empty	Domain must not be empty, locale must not be empty
Domain and locale	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)
Equivalence, domain, and locale	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)	Domain (Domain) locale (Domain) locale (Domain) locale (Domain)

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Mizar To Evo Language Delta Inventory (5/7). Point to the visible example, question, or diagram before moving on.

2.2 - Mizar To Evo Language Delta Inventory (6/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Schemes and generics	classical scheme, of/over parameterization	first-class templates; bracket form is canonical, constraints and inference are explicit, of/over are shorthands	unify parameterized definitions and theorem schemas without hiding instantiation choices
Declarative proof skeleton	Jaśkowski-style proof . . . end text with let, assume, thus, hence, thesis, and diffuse reasoning	preserve the readable proof skeleton and emit extracted obligations, thesis states, and source spans as metadata	protect Mizar's proof prose while making proof state toolable
Proof status	verified theorem items as the normal publication unit	open, assumed, and conditional theorem statuses	separate unsettled work, assumptions, and clean verified results

└ Part 2. Language Specification

2.2 - Mizar To Evo Language Delta Inventory (6/7)

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Mizar To Evo Language Delta Inventory (6/7). Point to the visible example, question, or diagram before moving on.

[illegible]

2.2 - Mizar To Evo Language Delta Inventory (7/7)

Area	Current Mizar baseline	Evo delta	Why it changes
Proof citations	by references to visible facts	grouped/bulk citations, used-axiom recording, citation refinement	support small verifier-checked repairs and AI-assisted edits
Type views	existing qua, implicit widening, and local disambiguation idioms	keep source-level qua, and emit resolved view/coercion metadata artifacts	make overload and inherited-view choices inspectable
Algorithms	mathematical proof language, not a general verified programming surface	algorithm, contracts, invariants, termination, MVM, by computation	support verified computation without changing theorem truth

Mizar Evo

└ Part 2. Language Specification

└ 2.2 - Mizar To Evo Language Delta Inventory (7/7)

Item	Current Mizar Language	New Delta	Mizar Language
Final object	Object name is still M1	Object name is still M1	Object name is still M1
Type name	Object name is still M1	Object name is still M1	Object name is still M1
Object	Object name is still M1	Object name is still M1	Object name is still M1

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Mizar To Evo Language Delta Inventory (7/7). Point to the visible example, question, or diagram before moving on.

2.3 - Mizar To Evo Artifact And Workflow Delta Inventory (1/3)

These rows are not purely syntactic, but they affect what migration tools, reviewers, editors, and packages can trust.

Area	Current Mizar baseline	Evo delta	Why it changes
Annotations	comments and informal tool hints	semantic-neutral annotations such as @proof_hint, @show_type, @show_resolution, @latex	guide tools while keeping logical meaning unchanged
Diagnostics and LSP	verifier messages are primarily human-facing output	stable diagnostic codes, primary/secondary spans, fix suggestions, lazy explanations, and LSP records	make failures actionable for humans, editors, and AI tools
ATP boundary	proof search and verifier behavior are not surfaced as a stable artifact boundary	ATP dispatch plus independently checked certificates and rejection categories	keep a small trusted base

Mizar Evo

└ Part 2. Language Specification

└ 2.3 - Mizar To Evo Artifact And Workflow Delta Inventory
(1/3)

These rows are not purely syntactic, but they affect what migration tools, reviewers,

Row	Comment: What breaks?	Row Delta	Why it breaks
Renamed	renamed the Mizar artifact	renamed the artifact	renamed the artifact
Unpackaged and LSP	unpackaged messages are already human-readable	unpackaged messages are already human-readable	unpackaged messages are already human-readable
APP boundary	unpackaged messages are already human-readable	unpackaged messages are already human-readable	unpackaged messages are already human-readable

Presenter script: Say:

- These are artifact deltas, but they belong near the language discussion because they determine whether a migrated source change remains explainable.

Read aloud:

- These rows are not purely syntactic, but they affect what migration tools,
- reviewers, editors, and packages can trust.

For the table, read the row labels first, then the design reason.

2.3 - Mizar To Evo Artifact And Workflow Delta Inventory (2/3)

Area	Current Mizar baseline	Evo delta	Why it changes
Packages and dependency resolution	article/MML workflow	mizar.pkg, lockfile, SemVer, features, compatibility checks, cached verifier artifacts	enable reproducible, package-oriented development
Incremental verification	accepted article output is the main reuse boundary	dependency slices, VC anchors, witness hashes, and cache keys; cache reuse is never proof authority	scale verification while preserving clean-build equivalence
Documentation generation	source comments and publication pages are separate from reusable API documentation	mizar doc, :::: doc comments, label/FQN cross-links, @latex, docs from verified artifacts	produce browsable API documentation without making docs a proof gate

Part 2. Language Specification

2.3 - Mizar To Evo Artifact And Workflow Delta Inventory (2/3)

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Mizar To Evo Artifact And Workflow Delta Inventory (2/3). Point to the visible example, question, or diagram before moving on.

[illegible]

2.3 - Mizar To Evo Artifact And Workflow Delta Inventory (3/3)

Area	Current Mizar baseline	Evo delta	Why it changes
Code extraction	no general verified extraction workflow in the source language	terminating algorithms, ghost erasure, target-neutral runtime IR, extractor configuration	keep executable output downstream of verified computation
Formalized Mathematics link	article and library identity are closely coupled	publication metadata links to reusable library modules	preserve citation value while supporting package reuse

Mizar Evo

└ Part 2. Language Specification

└ 2.3 - Mizar To Evo Artifact And Workflow Delta Inventory (3/3)

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Mizar To Evo Artifact And Workflow Delta Inventory (3/3). Point to the visible example, question, or diagram before moving on.

Item	Current Mizar To Evo	Design Reason	Design Reason
Item 1	as generalization of the previous language	the design is to be a generalization of the previous language	the design is to be a generalization of the previous language
Item 2	the design is to be a generalization of the previous language	the design is to be a generalization of the previous language	the design is to be a generalization of the previous language

Message:

- This chapter is the part we should not compress.
- The Bialystok review should treat the language surface as a specification, not only as a design story.
- Each syntax change should be reviewed for three things: readability for Mizar authors, migration cost for MML, and determinism for parsers, verifiers, and tools.
- The examples below are not final tutorial material. They are review targets: if a construct looks too unlike Mizar, or if a hidden Mizar idiom is missing, that is exactly the feedback we need.

Message:

- This chapter is the part we should not compress.
- The Bialystok review should treat the language surface as a specification, not only as a design story.
- Each syntax change should be reviewed for three things: readability for Mizar authors, migration cost for MML, and determinism for parsers, verifiers, and tools.
- The examples below are not final tutorial material. They are review targets: if a construct looks too unlike Mizar, or if a hidden Mizar idiom is missing, that is exactly the feedback we need.

Presenter script: Read aloud:

- This chapter is the part we should not compress.
- The Bialystok review should treat the language surface as a specification, not only as a design story.
- Each syntax change should be reviewed for three things: readability for Mizar authors, migration cost for MML, and determinism for parsers, verifiers, and tools.
- The examples below are not final tutorial material. They are review targets: if a construct looks too unlike Mizar, or if a hidden Mizar idiom is missing, that is exactly the feedback we need.

Canonical specification sources: All paths in this table are under `doc/spec/en/`.

Topic	Source
Cross-chapter grammar	<code>appendix_a.grammar_summary.md</code>
Operator precedence	<code>appendix_b.operator_precedence.md</code> , <code>10.functors.md</code>
Lexical structure	<code>02.lexical_structure.md</code>
Structures	<code>05.structures.md</code>
Attributes and modes	<code>06.attributes.md</code> , <code>07.modes.md</code>
Terms and formulas	<code>13.term_expression.md</code> , <code>14.formulas.md</code>
Statements and proofs	<code>15.statements.md</code> , <code>16.theorems_and_proofs.md</code>

└ Part 2. Language Specification

└ 2.5 - Grammar Sources For This Review (1/2)

Canonical specification sources: All paths in this table are under doc/spec/en/.

base	base
Contexts and grammar	01.contexts_and_grammar.md
Operator precedence	02.operator_precedence.md
Lexical structure	03.lexical_structure.md
Statements	04.statements.md
Attributes and modes	05.attributes_and_modes.md
Types and functions	06.types_and_functions.md
Global variables and constants	07.global_variables_and_constants.md

Presenter script: Read aloud:

- All paths in this table are under doc/spec/en/.

For the table, read the row labels first, then the design reason.

Slow down: this is language-specification review, not only implementation detail.

2.5 - Grammar Sources For This Review (2/2)

Topic	Source
Registrations and templates	<code>17.clusters_and_registrations.md</code> , <code>18.templates.md</code>

Review rule:

- The slides summarize the surface grammar. The spec files remain the authority for edge cases and well-formedness rules.

└ Part 2. Language Specification

└ 2.5 - Grammar Sources For This Review (2/2)

Topic	Location
Requirements and templates	02-25-grammar_and_templates.mli, 02-25-grammar.mli

Review rule:

- The slides summarize the surface grammar. The spec files remain the authority for edge cases and well-formedness rules.

Presenter script: Read aloud:

- The slides summarize the surface grammar. The spec files remain the authority for edge cases and well-formedness rules.

For the table, read the row labels first, then the design reason.

Slow down: this is language-specification review, not only implementation detail.

The review should cover this grammar stack in order:

- 1 lexical classes and active lexicon;
- 2 module/import prelude and item activation;
- 3 definition blocks and visibility;
- 4 type expressions, attributes, modes, and structures;
- 5 functors, predicates, notation, properties, and overload families;
- 6 operator precedence and term/formula boundary;
- 7 terms, formulas, comprehensions, the, sethood, and qua;

The review should cover this grammar stack in order:

- 1 theorem/proof statements and citations;
- 2 registrations, reductions, templates, annotations, and algorithms.

└ Part 2. Language Specification

└ 2.6 - Surface Syntax Review Map (1/2)

The review should cover this grammar stack in order:

- lexical classes and active lexicon;
- module/import prelude and item activation;
- definition blocks and visibility;
- type expressions, attributes, modes, and structures;
- functors, predicates, notations, properties, and overload families;
- operators: precedence and term/formula bounding;
- terms, formulas, comprehension, the, sethood, and qua;

The review should cover this grammar stack in order:

- theorem/proof statements and lemmas;
- regulations, redactions, templates, annotations, and algorithms.

Presenter script: Read aloud:

- lexical classes and active lexicon;
- module/import prelude and item activation;
- definition blocks and visibility;
- type expressions, attributes, modes, and structures;
- functors, predicates, notation, properties, and overload families;

Slow down: this is language-specification review, not only implementation detail.

Why this order matters:

- Mizar-style readability is not only local syntax. It depends on when names become visible, which notation is active, and which automation is allowed to fire.

Presenter script: Read aloud:

- Mizar-style readability is not only local syntax. It depends on when names become visible, which notation is active, and which automation is allowed to fire.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
compilation_unit ::= import_prelude export_prelude  
                  { annotated_declaration } ;  
  
import_prelude   ::= { import_stmt } ;  
export_prelude   ::= { export_stmt } ;  
  
declaration      ::= definition_block  
                  | reserve_decl  
                  | registration_block  
                  | claim_block
```

Grammar sketch:

```
                  | [ visibility ] theorem_item  
                  | [ visibility ] notation_decl ;  
  
visibility        ::= "private" | "public" ;
```

Grammar & block:

```

compilation_unit ::= import_prelude export_prelude
                  { statement_declaration } ;

import_prelude  ::= { import_item } ;
export_prelude  ::= { export_item } ;

declaration ::= declaration_block
             | function_decl
             | statement_block
             | class_block

```

Grammar & block:

```

| visibility | theorem_item
| visibility | definition_decl ;

visibility ::= "private" | "public" ;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Compilation Unit Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
import algebra.group as group;
export algebra.group;

public theorem left_cancel:
  for G being Group, a, b, c being Element of G
  st a * b = a * c holds b = c;
```

Review Point:

- The file begins with a reproducible dependency surface, then ordinary Mizar items. This replaces article-environment roles without turning the language into a generic programming module system.

Example:

```
import m3_group, group as group;
export m3_group, group;

public theorem left_cancell:
  for G being Group, m, b, c being Element of G
  ex m = b * m * c holds b = c;
```

Review Point:

- The file begins with a reproducible dependency surface, then ordinary Mizar items. This replaces article-environment roles without turning the language into a generic programming module system.

Presenter script: Read aloud:

- The file begins with a reproducible dependency surface, then ordinary Mizar items. This replaces article-environment roles without turning the language into a generic programming module system.

After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
import_stmt ::= "import" module_alias_decl  
              { ",", module_alias_decl } ";" ;  
export_stmt ::= "export" module_path { ",", module_path } ";" ;  
  
module_alias_decl ::= module_path [ "as" module_identifier ]  
                  | module_branch_import ;
```

Example:

```
import algebra.group as group;  
import algebra.ring.{Ring, Ideal};  
export algebra.group;
```

└ Part 2. Language Specification

└ 2.8 - Import Prelude And Item Activation (1/2)

Grammar sketch:

```
import_stmt ::= "import" module_name_decl  
            { ":", module_name_decl } ";" ;  
export_stmt ::= "export" module_name { ":", module_name } ";" ;  
module_name_decl ::= module_name [ "as" module_identifier ]  
                  module_name_decl import ;
```

Example:

```
import algebra.group as group;  
import algebra.ring_of_ring, idem;  
export algebra.group;
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Import Prelude And Item Activation (1/2). Point to the visible example, question, or diagram before moving on.

Semantic rule:

- All `import` statements must appear before the first non-import item.
- Imported symbols seed the active lexicon for the body.
- Declarations in the current module become active only after their item is complete.
- A later import-shaped statement is a syntax error, not a dynamic environment update.

Reason:

- This makes lexing, parsing, and AI context extraction deterministic.

Semantic rule:

- All import statements must appear before the first non-import item.
- Imported symbols seed the active lexicon for the body.
- Declarations in the current module become active only after their item is complete.
- A later import-shaped statement is a syntax error, not a dynamic environment update.

Reason:

- This makes lexing, parsing, and AI context extraction deterministic.

Presenter script: Read aloud:

- All import statements must appear before the first non-import item.
- Imported symbols seed the active lexicon for the body.
- Declarations in the current module become active only after their item is complete.
- A later import-shaped statement is a syntax error, not a dynamic environment update.
- This makes lexing, parsing, and AI context extraction deterministic.

Grammar sketch:

```
identifier      ::= ( letter | "_" )  
                  { letter | digit | "_" | "'" } ;  
constructor_name ::= identifier | readable_constructor_name ;  
user_symbol     ::= symbol_char { symbol_char } ;
```

Example:

```
struct ZeroStr where  
  field carrier -> set;  
end;  
  
func + (x, y: Integer) -> Integer equals int.add(x, y);
```

└ Part 2. Language Specification

└ 2.9 - Lexical Classes (1/2)

Grammar sketch:

```
ident id_char ::= { letter | "_" }  
{ letter | digit | "_" | "" } ;  
constructor_name ::= ident id_char | readable_constructor_name ;  
user_symbol ::= symbol_char { symbol_char } ;
```

Example:

```
axiom is true where  
  f is of carrier -> set;  
end;  
  
[func f (x, y: integer) => integer equals int.add(x, y);
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Lexical Classes (1/2). Point to the visible example, question, or diagram before moving on.

Policy:

- `mode`, `struct`, and `attr` names use constructor names.
- `field` and `property` names are identifiers.
- Arbitrary symbolic notation is concentrated in `func` and `pred`.
- Tokenization uses the longest active match at the source position.

Review Point:

- This is one of the largest visible syntax differences from legacy Mizar; it should be reviewed with real algebraic and set-theoretic examples.

Policy:

- mode, struct, and attr names use constructor names.
- field and property names are identifiers.
- Arbitrary symbolic notation is concentrated in func and pred.
- Tokenization uses the longest active match at the source position.

Review Point:

- This is one of the largest visible syntax differences from legacy Mizar; it should be reviewed with real algebraic and set-theoretic examples.

Presenter script: Read aloud:

- mode, struct, and attr names use constructor names.
- field and property names are identifiers.
- Arbitrary symbolic notation is concentrated in func and pred.
- Tokenization uses the longest active match at the source position.
- This is one of the largest visible syntax differences from legacy Mizar; it should be reviewed with real algebraic and set-theoretic examples.

Reserved punctuation includes:

```
, . .. ; : := ( ) [ ] { } .{ = <> & -> . = .* @[ ...
```

Dot policy:

- . can be selector syntax, namespace qualification, or a user functor symbol, depending on grammar position and the active lexicon.
- The parser preserves enough surface form for the resolver to classify it.

Bracket policy:

- [and] are reserved for template arguments and built-in bracket functor syntax.
- Postfix indexing such as `x[y]` is not silently admitted.

2.10 - Reserved Symbols And Dot Disambiguation (1/2)

a Postfix indexing such as `x[i]` is not silently admitted.

After the example, say which design obligation the syntax makes visible.

Example:

```
let R be Ring;  
set U = R.carrier;  
set e = algebra.group.identity(R qua Group);
```

Reason:

- Ambiguous punctuation must not make migration depend on parser guesses.

└ Part 2. Language Specification

└ 2.10 - Reserved Symbols And Dot Disambiguation (2/2)

Example:

```
let N be Ring;  
let U = N carrier;  
let s = migration_group, side_group, group;
```

Reason:

- Ambiguous punctuation must not make migration depend on parser guesses.

Presenter script: Read aloud:

- Ambiguous punctuation must not make migration depend on parser guesses.

After the example, say which design obligation the syntax makes visible.

Grammar sketch:

```
type_expression ::= attribute_chain type_head ;
type_head       ::= radix_type | mode_type ;

attribute_chain ::= { [ "non" ] attribute_ref } ;
attribute_ref   ::= [ param_prefix ]
                   [ struct_ref_name "." ] attribute_name
                   [ "(" argument_list ")" ] ;

radix_type      ::= "object" | "set"
                  | struct_ref_name [ type_args ] ;
```

Grammar sketch:

```
mode_type       ::= mode_ref_name [ type_args ] ;
```

└ Part 2. Language Specification

└ 2.11 - Type Expression Syntax (1/2)

```

Grammar sketch:
type_expr_name ::= identifier_name type_name ;
type_name ::= basic_type | made_type ;
identifier_name ::= C | 'foo' | identifier_ref ;
identifier_ref ::= { param_name :
                  | identifier_name '.', identifier_name
                  | '*' argument_name '*' } ;
basic_type ::= 'int' | 'real' ;
identifier_ref_name ::= type_name ;

```

```

Grammar sketch:
made_type ::= made_ref_name | type_name ;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Type Expression Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
reserve G for non empty Group;  
let x be Element of G;  
let H be commutative subgroup of G;
```

Review Point:

- Mizar's soft type style is preserved, but the grammar makes the split between radix type, mode type, and attribute chain explicit.

Example:

```
preserve G for non empty Group;  
let x be Element of G;  
let H be commutative subgroup of G;
```

Review Point:

- Mizar's soft type style is preserved, but the grammar makes the split between radix type, mode type, and attribute chain explicit.

Presenter script: Read aloud:

- Mizar's soft type style is preserved, but the grammar makes the split between radix type, mode type, and attribute chain explicit.

After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
definition_block ::= "definition"  
                  { definition_content }  
                  "end" ";" ;  
  
definition_content ::= { annotation }  
                    ( definition_parameter_decl  
                      | assumption  
                      | correctness_condition  
                      | property_item  
                      | [ visibility ] definitional_item
```

Grammar sketch:

```
                    | [ visibility ] theorem_item  
                    | [ visibility ] registration_item ) ;
```

```

Grammar <block>:
definition_block ::= "definition"
                  { definition_item_definition }
                  "end" ";" ;

definition_item_definition ::= { definition }
                           { example_definition }
                           { correction_definition }
                           { property_item }
                           { initialization_definition_item }

```

```

Grammar <block>:
| | v m d b l k y | theorem_item
| | v m d b l k y | registration_item | ;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Definition Block Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
definition
  let G be Group;
  func UnitDef: unit(G) -> Element of G
    means for x being Element of G holds it * x = x;
  existence by group.unit_exists;
  uniqueness by group.unit_unique;
end;
```

Review Point:

- The definition ... end envelope remains familiar.
- New surface elements should be judged by whether they clarify obligations without breaking Mizar's definition-oriented reading style.

Example:

```

def in G is in
  let G be Group;
  from the G's unit() -> Element of G
  mean for x being Element of G hold it = x * x;
  existence by group.unit_exists;
  uniqueness by group.unit_unique;
end;

```

Review Point:

- The `def` `is in` ... `end` envelope remains familiar.
- New surface elements should be judged by whether they clarify obligations without breaking Mizar's definition-oriented reading style.

Presenter script: Read aloud:

- The definition ... end envelope remains familiar.
- New surface elements should be judged by whether they clarify obligations without breaking Mizar's definition-oriented reading style.

After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
struct_def      ::= "struct" struct_def_name [ type_params ]  
                  "where" struct_member { struct_member }  
                  "end" ";" ;  
  
struct_member   ::= field_decl | property_decl ;  
field_decl      ::= "field" identifier "->" type_expression  
                  [ ":@" term_expression ] ";" ;  
property_decl   ::= "property" identifier "->" type_expression ";" ;
```

Example:

```
struct UnitalMagmaStr where  
  field carrier -> non empty set;  
  field mult -> BinOp of carrier;  
  property unit -> Element of carrier;  
end;
```

└ Part 2. Language Specification

└ 2.13 - Structure Declaration Syntax (1/2)

Grammar sketch:

```

struct_def ::= "struct" struct_def_name ( type_params )
           "where" struct_member ( struct_member )
           "end" ";" ;

struct_member ::= struct_name ( property_name )
struct_def ::= "struct" identifier ">" type_expression
           ( ";" struct_expression ";" ) ;
property_name ::= "property" identifier ">" type_expression ";" ;

```

Example:

```

structure ThisIsMergedOrWhere
  of kind carrier => non empty set;
  of kind mid1 => Union of carrier;
  property mid2 => Element of carrier;
end;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Structure Declaration Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- `field` declares intrinsic data of the structure value.
- `property` declares a canonical derived slot or obligation.
- The syntax deliberately separates layout from mathematically determined values.

- field declares intrinsic data of the structure value.
- property declares a canonical derived slot or obligation.
- The syntax deliberately separates layout from mathematically determined values.

Presenter script: Read aloud:

- field declares intrinsic data of the structure value.
- property declares a canonical derived slot or obligation.
- The syntax deliberately separates layout from mathematically determined values.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
property_means_impl  ::= "property" identifier "." identifier  
                        "means" formula_definiens ";"  
                        existence_block uniqueness_block ;  
property_equals_impl ::= "property" identifier "." identifier  
                        "equals" term_definiens ";" ;
```

Example:

```
definition  
  let M be UnitalMagma;  
  property M.unit means  
    for x being Element of M.carrier holds  
      M.mult(it, x) = x & M.mult(x, it) = x;  
  existence by magma.unit_exists;  
  uniqueness by magma.unit_unique;  
end;
```

└ Part 2. Language Specification

└ 2.14 - Property Implementation Syntax (1/2)

Grammar sketch:

```

property_name_impl ::= "property" identifier "." identifier
                    "has" formal_def_infix ";"
                    existence_block uniqueness_block ;
property_eqname_impl ::= "property" identifier "." identifier
                       "equals" term_def_infix ";" ;

```

Example:

```

definition
  let B be UnivAlg;
  property B.uniq: unique
  for a being Element of B.uniq holds
    B.uniq(a, a) = a & B.uniq(a, a) = a;
  uniqueness of B.uniq.uniq;
end;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Property Implementation Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- A property in a `struct` is a virtual slot, not stored field data.
- The implementation is supplied later for a mode whose attributes guarantee the required assumptions.
- `means` carries existence and uniqueness; `equals` directly supplies the value.

Review Point:

- A property in a struct is a virtual slot, not stored field data.
- The implementation is supplied later for a mode whose attributes guarantee the required assumptions.
- means carries existence and uniqueness; equals directly supplies the value.

Presenter script: Read aloud:

- A property in a struct is a virtual slot, not stored field data.
- The implementation is supplied later for a mode whose attributes guarantee the required assumptions.
- means carries existence and uniqueness; equals directly supplies the value.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
inherit_def ::= "inherit" inherit_child "extends" parent_type  
             [ "where" inherit_member { inherit_member }  
               [ coherence_block ] "end" ] ";" ;  
  
field_redef  ::= "field" identifier [ "->" type_expression ]  
               "from" ( identifier | "it" ) ";" ;  
  
property_redef ::= "property" identifier [ "->" type_expression ]  
                "from" identifier ";" ;
```

Example:

```
inherit GroupStr extends UnitalMagmaStr where  
  field carrier from carrier;  
  field mult from mult;  
  property unit from unit;  
  coherence by group.inherit_unital_magma;  
end;
```

└ Part 2. Language Specification

└ 2.15 - Inheritance Declaration Syntax (1/2)

Grammar sketch:

```
inherit_def ::= 'inherit' inherit_child 'extends' parent_type
              { 'where' inherit_member { inherit_member }
              ; coherence_block { 'and' ; } ; }

child_def ::= 'child' identifier { '-' type_expression ;
                                   'from' identifier { 'is' ; } ; }

property_rule ::= 'property' identifier { '-'> type_expression ;
                                             'from' identifier ; } ;
```

Example:

```
inherit U_GroupStr extends UnitalMagModule where
  f id carrier from carrier;
  f id mult from mult;
  property unital from unital;
  coherence by group, inherit_unital, magmul;
end;
```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Inheritance Declaration Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Each `inherit` statement names exactly one parent.
- Renaming, narrowing, and coherence are visible source items.
- Diamond inheritance becomes a checked mapping problem rather than a hidden merge.

Review Point:

- Each inherit statement names exactly one parent.
- Renaming, narrowing, and coherence are visible source items.
- Diamond inheritance becomes a checked mapping problem rather than a hidden merge.

Presenter script: Read aloud:

- Each inherit statement names exactly one parent.
- Renaming, narrowing, and coherence are visible source items.
- Diamond inheritance becomes a checked mapping problem rather than a hidden merge.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
attr_def ::= "attr" label ":" subject "is" attr_pattern
           "means" formula_definiens ";" ;

mode_def ::= "mode" label ":" mode_def_name [ type_params ]
           "is" attribute_chain radix_type ";"
           [ mode_property ] ;
```

Example:

```
attr AssocDef:
  G is associative means
    for x, y, z being Element of G holds (x * y) * z = x * (y * z);

mode GroupDef: Group is associative invertible UnitalMagmaStr;
```

Grammar sketch:

```

attr_def ::= "attr" label ":" subject "is" attr_pattern
          "means" formal_def clause ":" ;

mode_def ::= "mode" label ":" mode_def_name : type_name {
          "is" attribute_chain mode_type ":"
          } mode_property ;

```

Example:

```

attr AssocDef:
  G is associative means
    for x, y, z being Element of G holds (x + y) + z = x + (y + z);

mode GroupDef: Group is associative invertible UnitRegister;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Attribute And Mode Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Attributes are unary type-refining predicates.
- Modes name reusable type classifications.
- This keeps Mizar's adjective-rich prose while making cluster participation explicit.

Review Point:

- Attributes are unary type-refining predicates.
- Modes name reusable type classifications.
- This keeps Mizar's adjective-rich prose while making cluster participation explicit.

Presenter script: Read aloud:

- Attributes are unary type-refining predicates.
- Modes name reusable type classifications.
- This keeps Mizar's adjective-rich prose while making cluster participation explicit.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
pred_def ::= "pred" label ":" pred_pattern
           "means" formula_definiens ";" ;

func_def ::= "func" label ":" func_pattern "->" type_expression
           ( "means" formula_definiens
             | "equals" term_definiens ) ";" ;
```

Example:

```
pred DividesDef:
  x divides y means ex z being Integer st y = x * z;

func InvDef: inverse(G, x) -> Element of G
  means x * it = unit(G) and it * x = unit(G);
```

└ Part 2. Language Specification

└ 2.17 - Predicate And Functor Syntax (1/2)

Grammar sketch:

```

pred_def ::= "pred" l_label ":" "pred_predicate"
           "meaning" formula_def in_line ";" ;

func_def ::= "func" l_label ":" "func_predicate" "<=>" type_expression in
           | "name" formula_def in_line
           | "equals" term_def in_line | ";" ;

```

Example:

```

pred DividesDef :
  x divides y means x <= being Integer & y = x * z ;

func InvDef : inverse(G, x) => Element of G
  means x * it = unit(G) and it * x = unit(G) ;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Predicate And Functor Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Definition patterns:

- Predicate and functor patterns may use symbolic or phrase notation.
- Their symbols become active only after the declaration item is complete.
- Overloaded definitions are resolved by roots, argument types, and refinement rules, not by accidental source-order behavior.

Definition patterns:

- Predicate and functor patterns may use symbolic or phrase notation.
- Their symbols become active only after the declaration item is complete.
- Overloaded definitions are resolved by roots, argument types, and refinement rules, not by accidental source-order behavior.

Presenter script: Read aloud:

- Predicate and functor patterns may use symbolic or phrase notation.
- Their symbols become active only after the declaration item is complete.
- Overloaded definitions are resolved by roots, argument types, and refinement rules, not by accidental source-order behavior.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
synonym_def  ::= "synonym" alt_pattern "for" original_pattern ";" ;
antonym_def  ::= "antonym" alt_pattern "for" original_pattern ";" ;

redefine_attr ::= "redefine" "attr" label ":" subject "is" attr_pattern
                  "means" formula_definiens ";"
                  "coherence" [ "with" label ] justification ";" ;
redefine_pred ::= "redefine" "pred" label ":" pred_pattern
                  "means" formula_definiens ";"
                  "coherence" [ "with" label ] justification ";" ;
redefine_func ::= "redefine" "func" label ":" func_pattern
```

Grammar sketch:

```
"->" type_expression
( "means" formula_definiens | "equals" term_definiens ) ";"
"coherence" [ "with" label ] justification ";" ;
```

└ Part 2. Language Specification

└ 2.18 - Synonyms, Antonyms, And redefine (1/3)

Grammar sketch:

```

syntagma_400 ::= "syntagma" ant_precedence "ant" syntagma_precedence "!" ;
antisyntagma_400 ::= "antisyntagma" ant_precedence "ant" syntagma_precedence "!" ;

on antonym_utter ::= "antonym" "utter" "utter" "!" subject "ant" utter_precedence
    "antonym" formal_definition "!" ;
"color name" : "rich" label : justification "!" ;

on antonym_poss ::= "antonym" "poss" "poss" "!" poss_precedence
    "antonym" formal_definition "!" ;
"color name" : "rich" label : justification "!" ;

on antonym_time ::= "antonym" "time" "time" "!" time_precedence
    "antonym" formal_definition "!" ;

```

Grammar sketch:

```

">" type_expression
: "name" for antonym_definition | "qualifier" term_definition "!" ;
"color name" : "rich" label : justification "!" ;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Synonyms, Antonyms, And redefine (1/3). Point to the visible example, question, or diagram before moving on.

Example:

```
definition
  let a, b be Real;
  pred LessDef: a < b means real.lt(a, b);
  synonym b > a for a < b;
  antonym a >= b for a < b;
end;

definition
  let x be non negative Real;
  redefine func AbsNonNeg: |.x.| -> non negative Real equals x;
```

Example:

```
  coherence with AbsGeneral by real.abs_nonneg;
end;
```

Example:

```

definition
  let a, b be Real;
  prec: lemma: a < b means result(a, b);
  synonym a > b for a < b;
  synonym a >= b for a < b;
end;

definition
  let a be non negative Real;
  redefine sym: synonym: [-a, 1] => non negative Real equals a;
end;

```

Example:

```

coherence with AbsGeneral by real.abs, commg;
end;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Synonyms, Antonyms, And redefine (2/3). Point to the visible example, question, or diagram before moving on.

Review Point:

- Synonyms and antonyms preserve mathematical phrasing and natural negation.
- They are semantic aliases, not new roots.
- `redefine` sharpens the same root with `coherence with`; it is not a rival overload.

- Synonyms and antonyms preserve mathematical phrasing and natural negation.
- They are semantic aliases, not new roots.
- `redefine` sharpens the same root with coherence with; it is not a dual overload.

Presenter script: Read aloud:

- Synonyms and antonyms preserve mathematical phrasing and natural negation.
- They are semantic aliases, not new roots.
- `redefine` sharpens the same root with coherence with; it is not a rival overload.

Grammar sketch:

```
pred_property ::= ( "symmetry" | "asymmetry" | "connectedness"  
                  | "reflexivity" | "irreflexivity" )  
                justification ";" ;  
  
func_property ::= ( "commutativity" | "idempotence"  
                  | "involutiveness" | "projectivity" )  
                justification ";" ;
```


Grammar sketch:

```

pred_property ::= | "symmetry" | "asymmetry" | "commutativity"
               | "reflexivity" | "irreflexivity" |
               | "justification" ;

func_property ::= | "commutativity" | "idempotence"
                 | "associativity" | "projectivity" |
                 | "justification" ;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Predicate And Functor Properties (1/3). Point to the visible example, question, or diagram before moving on.

Example:

```

definition
  let x, y be Real;
  pred LessDef: x < y means x is_less_than y;
  asymmetry by real.lt_asym;
  irreflexivity by real.lt_irrefl;
end;

```

```

definition
  let X, Y be set;
  func UnionDef: X \/ Y -> set means

```

Example:

```

  for z being object holds z in it iff z in X or z in Y;
  commutativity by set.union_comm;
  idempotence by set.union_idem;
end;

```

Example:

```

definition
  let X, Y be Set;
  pred is_included: x < y means x = y, then y;
  asymmetry by real_01, asym;
  reflexivity by real_01, reflexivity;
end;

definition
  let X, Y be set;
  func UnionDef: X \ Y -> set means

```

Example:

```

  for x being object holds x in X iff x in X or x in Y;
  commutativity by set, union_1, sym;
  idempotence by set, union_2, idem;
end;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Predicate And Functor Properties (2/3). Point to the visible example, question, or diagram before moving on.

Review Point:

- Properties are not comments and not trusted hints.
- Each property is accepted only with a proof obligation.
- The design keeps algebraic notation compact while recording exactly which automatic matching, rewriting, and ATP facts may be used.

Review Point:

- Properties are not comments and not trusted hints.
- Each property is accepted only with a proof obligation.
- The design keeps algebraic notation compact while recording exactly which automatic matching, rewriting, and ATP facts may be used.

Presenter script: Read aloud:

- Properties are not comments and not trusted hints.
- Each property is accepted only with a proof obligation.
- The design keeps algebraic notation compact while recording exactly which automatic matching, rewriting, and ATP facts may be used.

Key distinction:

Form	Meaning	Obligations
<code>equals</code>	gives a direct term definition	usually immediate well-definedness
<code>means</code>	characterizes a value or relation	may require existence, uniqueness, coherence, or compatibility
<code>assume</code>	adds a local definition-side condition	must be recorded as part of the generated obligation

Grammar sketch:

```

existence_block      ::= "existence" justification ";" ;
uniqueness_block     ::= "uniqueness" justification ";" ;
coherence_block      ::= "coherence" justification ";" ;
compatibility_block  ::= "compatibility" justification ";" ;

```

└ Part 2. Language Specification

└ 2.20 - equals, means, And Correctness Blocks (1/2)

Key distinction:		
Term	Meaning	Comments
exists	exists a fixed set of objects	used to introduce an existence
forall	characterizes a value or values	used to require a sentence, or a sequence, or a sequence, or a sequence
assume	adds a local restriction on objects	must be recorded as part of the generated obligation

Grammar sketch:	
existence_block	:= "existence" just if local in " ; " ;
forall_block	:= "forall" just if local in " ; " ;
assumption_block	:= "assume" just if local in " ; " ;
compatibility_block	:= "compatibility" just if local in " ; " ;

Presenter script: After the example, say which design obligation the syntax makes visible.

For the table, read the row labels first, then the design reason.

Use this as the transition: equals, means, And Correctness Blocks (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
func InvDef: inverse(G, x) -> Element of G
  means x * it = unit(G) and it * x = unit(G);
existence by group.inverse_exists;
uniqueness by group.inverse_unique;
```

Review Point:

- The syntax should preserve Mizar's definitional idiom while exposing the proof obligations that migration tools must not hide.

└ Part 2. Language Specification

└ 2.20 - equals, means, And Correctness Blocks (2/2)

Example:

```

{
  func Involved : involved(G, x) => Element of G
  means x < x < inv(G) and it < x < inv(G);
  existence by group_involved_exists;
  uniqueness by group_involved_unique;
}

```

Review Point:

- The syntax should preserve Mizar's definitional idiom while exposing the proof obligations that migration tools must not hide.

Presenter script: Read aloud:

- The syntax should preserve Mizar's definitional idiom while exposing the proof obligations that migration tools must not hide.

After the example, say which design obligation the syntax makes visible.

Grammar sketch:

```
infix_operator_decl  ::= "infix_operator" "(" string_literal ","  
                        infix_assoc "," nat_literal ")" ";" ;  
prefix_operator_decl ::= "prefix_operator" "(" string_literal ","  
                        nat_literal ")" ";" ;  
postfix_operator_decl ::= "postfix_operator" "(" string_literal ","  
                        nat_literal ")" ";" ;  
infix_assoc          ::= "left" | "right" | "none" ;
```

└ Part 2. Language Specification

└ 2.21 - Operator Declarations And Precedence (1/3)

Grammar sketch:

```
inf ix_op precedence_decl ::= "inf ix_op precedence" "[" string_literal "]"  
inf ix_op precedence ::= "inf ix_op precedence" "[" string_literal "]"  
pre def op precedence_decl ::= "pre def op precedence" "[" string_literal "]"  
pre def op precedence ::= "pre def op precedence" "[" string_literal "]"  
pre def op precedence_decl ::= "pre def op precedence" "[" string_literal "]"  
pre def op precedence ::= "pre def op precedence" "[" string_literal "]"  
inf ix_op precedence ::= "inf ix_op precedence" "[" string_literal "]"
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Operator Declarations And Precedence (1/3). Point to the visible example, question, or diagram before moving on.

Example:

```
infix_operator("+", left, 80);  
infix_operator("*", left, 90);  
infix_operator("^", right, 95);  
prefix_operator("-", 85);  
postfix_operator("!", 95);
```

$a + b * c$:: $a + (b * c)$

$a ^ b ^ c$:: $a ^ (b ^ c)$

$a \% b \% c$:: error if $\%$ is non-associative

Part 2. Language Specification

2.21 - Operator Declarations And Precedence (2/3)

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Operator Declarations And Precedence (2/3). Point to the visible example, question, or diagram before moving on.

```

indis_operator! '=', left, 001;
indis_operator! '=', left, 001;
indis_operator! '=', right, 001;
preous_operator! '-', 001;
preous_operator! '-', 001;

```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99

Design Reason:

- Term precedence and formula precedence are separate domains.
- Precedence values range from 0 to 255; higher values bind more tightly.
- A symbolic functor with no declaration defaults to precedence 64 and non-associative grouping.
- Operator declarations affect only later tokens; they are not forward declarations, and the target spelling must already be an active functor.
- Parsing chooses grouping from operator metadata; overload resolution later chooses the semantic root.
- Conflicting imported metadata for the same visible user symbol is a link-time conflict.
- Built-in predicate symbols such as `=`, `<>`, and `in` come from the core module and cannot be overridden.

Design Reason:

- `qua` is fixed by the language, has the lowest term precedence, and is left-associative.

Design Reason:

- Term precedence and formula precedence are separate domains.
- Precedence values range from 0 to 255; higher values bind more tightly.
- A symbolic functor with no declaration defaults to precedence 64 and non-associative grouping.
- Operator declarations affect only later tokens; they are not forward declarations, and the target spelling must already be an active functor.
- Parsing chooses grouping from operator metadata; overload resolution later chooses the semantic root.
- Conflicting imported metadata for the same symbol is a lifetime conflict.
- Built-in predicate symbols such as $=$, $<$, $>$, and $\in$ come from the core module and cannot be overridden.

Design Reason:

- qua is fixed by the language, has the lowest term precedence, and is left-associative.

Presenter script: Read aloud:

- Term precedence and formula precedence are separate domains.
- Precedence values range from 0 to 255; higher values bind more tightly.
- A symbolic functor with no declaration defaults to precedence 64 and non-associative grouping.
- Operator declarations affect only later tokens; they are not forward declarations, and the target spelling must already be an active functor.
- Parsing chooses grouping from operator metadata; overload resolution later chooses the semantic root.

Grammar sketch:

```

term_expression ::= operator_expression
                  { "qua" type_expression } ;

term_postfix     ::= "." field_name [ "(" [ term_list ] ")" ]
                  | "with" "(" field_update_list ")" ;

field_update     ::= selector ":@" term_expression ;

```

Example:

```

let x be object;
reconsider y = x qua Element of G;
set C = G.carrier;
set H = G with (carrier := C);

```


Grammar sketch:

```

term_expression ::= object_term
                  { "qua" type_expression } ;

term_prefix ::= "." d_term
             { "with" "}" d_term_update_list "}" ;

d_term_update ::= selector ":" term_expression ;

```

Example:

```

let x be object;
reconsider y = x qua Element of G;
let G = G.number;
let H = G with carrier := G;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Term Expressions, Selectors, And qua (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Field/property access remains readable.
- qua remains source-level syntax because it records an intended view, not a failed inference.
- Selector syntax and namespace syntax must be resolved with source spans and explanation artifacts.

└ Part 2. Language Specification

└ 2.22 - Term Expressions, Selectors, And qua (2/2)

Review Point:

- Field/property access remains readable.
- qua remains source-level syntax because it records an intended view, not a failed inference.
- Selector syntax and namespace syntax must be resolved with source spans and explanation artifacts.

Presenter script: Read aloud:

- Field/property access remains readable.
- qua remains source-level syntax because it records an intended view, not a failed inference.
- Selector syntax and namespace syntax must be resolved with source spans and explanation artifacts.

Grammar sketch:

```
term_primary      ::= variable_identifier  
                  | "it"  
                  | numeral  
                  | "(" term_expression ")"  
                  | struct_constructor  
                  | set_expression  
                  | choice_expression  
                  | inline_functor_application  
                  | template_functor_application  
                  | bracket_functor_application ;
```

└ Part 2. Language Specification

└ 2.23 - Term Primary Forms: Constructors, Sets, And the (1/3)

Grammar sketch:

```
term_primary ::= "(" term_primary ")"
              | "set"
              | "constructor"
              | "set_of"
              | "set_of_constructor"
              | "set_of_constructor_of"
              | "set_of_constructor_of_constructor"
              | "set_of_constructor_of_constructor_of_constructor"
              | "set_of_constructor_of_constructor_of_constructor_of_constructor"
              | "set_of_constructor_of_constructor_of_constructor_of_constructor_of_constructor"
              | "set_of_constructor_of_constructor_of_constructor_of_constructor_of_constructor_of_constructor"
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Term Primary Forms: Constructors, Sets, And the (1/3). Point to the visible example, question, or diagram before moving on.

Grammar sketch:

```

struct_constructor ::= struct_name [ type_args ]
                    "(" [ field_assignment_list ] ")" ;
inline_functor_application ::= inline_func_name "(" [ term_list ] ")" ;
template_functor_application ::= functor_symbol template_args
                               [ "(" [ term_list ] "]" ] ;
bracket_functor_application ::= user_symbol term_list user_symbol
                              | "[" term_list "]" ;
set_expression      ::= "{" [ term_list ] "}"
                     | "{" term_expression "where"

```

Grammar sketch:

```

                    typed_var_list [ ":" formula ] "}" ;
choice_expression ::= "the" type_expression ;

```

Grammar sketch:

```

term_constructor ::= constructor_name [ type_argument ]
                  | "(" [ field_assignment_list ] ")" ;
initial_constructor_application ::= initial_constructor_name "(" term_list ")" ;
complex_constructor_application ::= constructor_name [ type_argument ] "(" term_list ")" ;
bracketed_constructor_application ::= "(" symbol term_list symbol ")" ;
term_expression ::= "(" term_list ")" ;
                  | "<" term_expression "above"

```

Grammar sketch:

```

typed_term_list ::= "(" domain ")" ;
chained_expression ::= "the" type_expression ;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Term Primary Forms: Constructors, Sets, And the (2/3). Point to the visible example, question, or diagram before moving on.

Example:

```
set p = Point(x: 3, y: 4);  
set evens = { n where n is Element of NAT : n mod 2 = 0 };  
set witness = the Element of X;
```

Review Point:

- These are ordinary mathematical terms, not algorithmic runtime constructs.
- the T depends on non-emptiness evidence for T.
- Set comprehensions are admitted only when the generator range is known to be a set.

Example:

```

set p = Pairs(x: 3, y: 4);
set even = { n where n is Even and NAT : n mod 2 = 0 };
set squares = { n | Even n, n < 10 };

```

Review Point:

- These are ordinary mathematical terms, not algorithmic runtime constructs.
- that T depends on non-emptiness evidence for T .
- Set comprehensions are admitted only when the generator range is known to be a set.

Presenter script: Read aloud:

- These are ordinary mathematical terms, not algorithmic runtime constructs.
- the T depends on non-emptiness evidence for T .
- Set comprehensions are admitted only when the generator range is known to be a set.

After the example, say which design obligation the syntax makes visible.

Grammar sketch:

```
mode_property      ::= "sethood" justification ";" ;  
set_comprehension ::= "{" term_expression "where"  
                        typed_var_list [ ":" formula ] "}" ;
```

Example:

```
definition  
  mode FiniteOrdinalDef: FiniteOrdinal is ordinal finite set;  
  sethood by ordinal.finite_ordinal_sethood;  
end;  
  
set S = { f where f is FiniteOrdinal : f is non empty };
```

└ Part 2. Language Specification

└ 2.24 - sethood And Fraenkel Safety (1/2)

Grammar sketch:

```
node_property    := "sethood" justification "I";  
set_comprehension := "{ " term_1 comma in " where "  
                    typed_var_1 int | " formula | " }";
```

Example:

```
def is finite in  
  make FiniteOrdinDef: FiniteOrdinDef is ordinal finite set;  
  sethood by ordinal.finite_ordinal_sethood;  
end;  
  
set S = { d where d is FiniteOrdinDef : d is non empty };
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: sethood And Fraenkel Safety (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- sethood proves that all instances of a mode form a set, not a proper class.
- The type checker rejects $\{ f(x) \text{ where } x \text{ is } T : P(x) \}$ when no sethood witness or element-of-set range is available.
- This keeps the Tarski-Grothendieck set-theoretic boundary visible before ATP proof search begins.

Review Point:

- sethood proves that all instances of a mode form a set, not a proper class.
- The type checker rejects $\{ f(x) \text{ where } x \text{ is } T : P(x) \}$ when no sethood witness or element-of-set range is available.
- This keeps the Tarski-Grothendieck set-theoretic boundary visible before ATP proof search begins.

Presenter script: Read aloud:

- sethood proves that all instances of a mode form a set, not a proper class.
- The type checker rejects $\{ f(x) \text{ where } x \text{ is } T : P(x) \}$ when no sethood witness or element-of-set range is available.
- This keeps the Tarski-Grothendieck set-theoretic boundary visible before ATP proof search begins.

Grammar sketch:

```
formula          ::= quantified_formula | iff_formula ;
universal_formula ::= "for" quantified_vars [ "st" formula ]
                  ( "holds" formula | quantified_formula ) ;
existential_formula ::= "ex" quantified_vars "st" formula ;

iff_formula       ::= implies_formula
                  [ "iff" ( implies_formula | quantified_formula ) ] ;
implies_formula   ::= or_formula
                  [ "implies" ( implies_formula | quantified_formula ) ] ;
or_formula        ::= and_formula
```

Grammar sketch:

```

formula      ::= quantified_formula | iff_formula ;
universal_formula ::= 'forall' quantified_part 'is' formula ;
              ::= 'forall' formula | quantified_formula ;
existential_formula ::= 'exists' quantified_part 'is' formula ;

iff_formula   ::= implies_formula
              ::= 'iff' | implies_formula | quantified_formula ;
implies_formula ::= <formula>
              ::= 'implies' | implies_formula | quantified_formula ;
or_formula    ::= or_formula

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Formula Syntax (1/4). Point to the visible example, question, or diagram before moving on.

Grammar sketch:

```

                                { "or" ( and_formula | quantified_formula )
                                | "or" "... " "or"
                                  ( and_formula | quantified_formula ) } ;
and_formula      ::= not_formula
                                { "&" ( not_formula | quantified_formula )
                                | "&" "... " "&"
                                  ( not_formula | quantified_formula ) } ;
not_formula     ::= "not" ( not_formula | quantified_formula )
                  | atomic_formula
                  | "(" formula ")"

```


Grammar sketch:

```

    ( "x" | and_formula | quantified_formula |
      | "x" ... "y"
      | and_formula | quantified_formula ) > ;
and_formula ::= not_formula
              ( "x" | not_formula | quantified_formula |
                | "x" ... "y"
                | not_formula | quantified_formula ) > ;
not_formula ::= "not" | not_formula | quantified_formula |
              | not_formula
              | "!" formula "!"

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Formula Syntax (2/4). Point to the visible example, question, or diagram before moving on.

Grammar sketch:

```

                                | "contradiction"
                                | "thesis" ;

atomic_formula      ::= predicate_application
                    | inline_predicate_application
                    | is_assertion ;

is_assertion        ::= term_expression "is" [ "not" ]
                    is_assertion_body ;

```

Example:

```

for x being Element of G st x is invertible holds
  ex y being Element of G st x * y = unit(G);

```

Grammar sketch:

```

| 'expression'
| 'thesis' ;

atomic_formula ::= prenex_preapplication
                | inductive_preapplication
                | is_application ;
is_application ::= term_expression 'is' | 'not' |
                is_application_not ;

```

Example:

```

for x being Element of G at x is invertible holds
  ex y being Element of G at x * y = unit[G];

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Formula Syntax (3/4). Point to the visible example, question, or diagram before moving on.

Review Point:

- Mizar-style quantified prose is preserved.
- Attribute assertions and type assertions intentionally share surface syntax until resolution classifies them.
- Repeated connective forms such as `& ... &` and `or ... or` are explicit formula syntax, not parser accidents.

Review Point:

- Mizar-style quantified prose is preserved.
- Attribute assertions and type assertions intentionally share surface syntax until resolution classifies them.
- Repeated connective forms such as $\& \dots \&$ and $\text{or} \dots \text{or}$ are explicit formula syntax, not parser accidents.

Presenter script: Read aloud:

- Mizar-style quantified prose is preserved.
- Attribute assertions and type assertions intentionally share surface syntax until resolution classifies them.
- Repeated connective forms such as $\& \dots \&$ and $\text{or} \dots \text{or}$ are explicit formula syntax, not parser accidents.

Slow down: this is language-specification review, not only implementation detail.

Precedence domains:

Domain	Source of precedence	Review point
Terms	user operator metadata plus fixed term syntax	parse before overload
Atomic formulas	predicates, equality, membership, is assertions	bridge from terms to formulas
Formulas	fixed hierarchy for not, &, or, implies, iff, quantifiers	no user-defined formula precedence

Part 2. Language Specification

2.26 - Term/Formula Boundary And Formula Precedence (1/3)

Precedence domains:		
Domain	Formula of precedence	Domain name
Term	any operator or variable plus fixed term names	primary term structure
Atomic formulas	predicates, equality, membership, is a question	bridge from term to formula
Formulas	fixed hierarchy of not, &, or, implies, if & quantifiers	predefined formula precedence

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Term/Formula Boundary And Formula Precedence (1/3). Point to the visible example, question, or diagram before moving on.

Example:

```
a + b = c + d
:: atomic formula over two parsed terms

not x > 0 & y > 0
:: (not (x > 0)) & (y > 0)

a or b implies c
:: (a or b) implies c

a iff b iff c
```

Example:

```
:: error: iff is non-associative
```


Example:

```
a = b * c + d
:: atomic_formula atom (a b c d)
end
a > b + c > d
:: term (a > b) (b > c) (c > d)
end
a or b implies c
:: (a or b) implies c
end
a and b and c
```

Example:

```
:: axiom: iff (a and b) and c and d
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Term/Formula Boundary And Formula Precedence (2/3). Point to the visible example, question, or diagram before moving on.

Review Point:

- Atomic formulas are the boundary between term parsing and formula parsing.
- Predicate-chain notation such as $a < b < c$ is resolved at this boundary, not as term-operator associativity.
- This makes the grouping explainable in diagnostics and stable for AI edits.

Review Points:

- Atomic formulas are the boundary between term parsing and formula parsing.
- Predicate-chain notation such as $a < b < c$ is resolved at this boundary, not as term-operator associativity.
- This makes the grouping explainable in diagnostics and stable for AI edits.

Presenter script: Read aloud:

- Atomic formulas are the boundary between term parsing and formula parsing.
- Predicate-chain notation such as $a < b < c$ is resolved at this boundary, not as term-operator associativity.
- This makes the grouping explainable in diagnostics and stable for AI edits.

Grammar sketch:

```

theorem_item    ::= [ theorem_status ] theorem_role
                  label_identifier ":" formula
                  [ justification ] ";" ;

theorem_status  ::= "open" | "assumed" | "conditional" ;
theorem_role    ::= "theorem" | "lemma" ;

```

Example:

```

open theorem cancellation_candidate:
  for a, b, c being Element of G st a * b = a * c holds b = c;

assumed lemma choice_profile:
  for X being non empty set holds ex x being object st x in X;

```

Grammar sketch:

```

theorem_item  ::= { theorem_status { theorem_item
                    label_item id sep ":" domain
                    { justification } "*" ;
theorem_status ::= "open" | "assumed" | "conditional" ;
theorem_rule  ::= "theorem" | "lemma" ;

```

Example:

```

open theorem cancellation_cancellation:
  for a, b, c being Element of G at a * b = a * c holds b = c;

assumed lemma cancellation:
  for X being non empty set holds ex x being object at x in X;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Theorem Item And Status Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- `open`, `assumed`, and `conditional` are source-visible because unfinished or policy-controlled material must not look like ordinary verified theorem material.
- Publication profiles can restrict which statuses are allowed.

- open, assumed, and conditional are source-visible because unfinished or policy-controlled material must not look like ordinary verified theorem material.
- Publication profiles can restrict which statuses are allowed.

Presenter script: Read aloud:

- open, assumed, and conditional are source-visible because unfinished or policy-controlled material must not look like ordinary verified theorem material.
- Publication profiles can restrict which statuses are allowed.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```
proof      ::= "proof" reasoning "end" ;
reasoning  ::= { annotated_statement } ;
statement  ::= [ "then" ] linkable_statement
              | standalone_statement ;

conclusion ::= ( "thus" | "hence" ) proposition
              justification ";" ;
```

Example:

```
proof
  let x be Element of G;
  assume A1: x = unit(G);
  thus thesis by A1, group.unit_left;
end;
```


Grammar sketch:

```

proof      ::= "proof" term using "end" ;
proving    ::= < formula > _statement > ;
statement  ::= [ "% has" ] ! < formula > _statement
              | at end < formula > _statement ;
conclusion ::= [ "% has" ] "hence" | "propositional"
              just if < formula > ";" ;

```

Example:

```

proof
  let x be Element of G;
  assume A1: x = unit G;
  then thus is by A1, group.unit_left;
end;

```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Proof Statement Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Preserved proof vocabulary:

- let, assume, given, consider, take;
- thus, hence, thesis, by;
- now ... end, hereby ... end, per cases, case, suppose;
- deffunc, defpred, and reconsider.

Review Point:

- The readable Jaśkowski skeleton is not optional. The new artifacts support tools, but the source proof should still read as Mizar mathematics.

Preserved proof vocabulary:

- let, assume, given, consider, take;
- thus, hence, thesis, by;
- now ... end, hereby ... end, per cases, case, suppose;
- deffunc, defpred, and reconsider.

Review Point:

- The readable Jaśkowski skeleton is not optional! The new artifacts support tools, but the source proof should still read as Mizar mathematics.

Presenter script: Read aloud:

- let, assume, given, consider, take;
- thus, hence, thesis, by;
- now ... end, hereby ... end, per cases, case, suppose;
- deffunc, defpred, and reconsider.
- The readable Jaśkowski skeleton is not optional. The new artifacts support tools, but the source proof should still read as Mizar mathematics.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```

iterative_equality ::= [ label_identifier ":" ]
                      term_expression "=" term_expression
                      simple_justification
                      ".*" term_expression simple_justification
                      { ".*" term_expression simple_justification } ";" ;

statement ::= [ "then" ] linkable_statement
            | standalone_statement ;

```

Example:

```

A1: f.(x + y) = f.x + f.y by Additive
    . = g.x + f.y by A2
    . = g.x + g.y by A3;

assume A2: f = g;
then f.x + f.y = g.x + f.y by FuncEq;

```

Grammar sketch:

```

iteration_statement ::= { label, condition } { '}'
                    ::= 'do' expression 'while' expression
                    ::= 'do' expression 'until' condition
                    ::= 'do' expression 'while' condition > '}' ;

statements ::= { 'then' } condition statements
           ::= statements ;

```

Example:

```

A1: f.x + y = f.x + f.y by Additivity
    * g.x = f.y by A1
    * g.x = g.y by A3;

assumes A2: f = g;
then f.x + f.y = g.x + f.y by PlusEq;

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Proof Statement Details (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- `.=` is a justified equality chain, not assignment.
- `then` records dependence on the immediately preceding statement.
- Statement-level `such that` and `and` create labeled assumptions, while formula-level `st` and `&` remain inside one formula.

└ Part 2. Language Specification

└ 2.29 - Proof Statement Details (2/2)

Review Point:

- $\dot{=}$ is a justified equality chain, not assignment.
- then records dependence on the immediately preceding statement.
- Statement-level such that and and create labeled assumptions, while formula-level st and $\&$ remain inside one formula.

Presenter script: Read aloud:

- $\dot{=}$ is a justified equality chain, not assignment.
- then records dependence on the immediately preceding statement.
- Statement-level such that and and create labeled assumptions, while formula-level st and $\&$ remain inside one formula.

Grammar sketch:

```
justification ::= simple_justification | proof | computation_proof ;
simple_justification ::= [ "by" references ] ;

reference ::= label_identifier [ template_args ]
            | qualified_reference [ template_args ]
            | grouped_reference
            | bulk_reference ;

grouped_reference ::= namespace_path ".{"
                    grouped_item { "," grouped_item } "}" ;
```

Grammar sketch:

```
bulk_reference    ::= namespace_path ".*" ;
```


2.30 - Citation And Reference Syntax (1/2)

[illegible]

Presenter script: After the example, say which design obligation the syntax makes visible. Slow down: this is language-specification review, not only implementation detail. Use this as the transition: Citation And Reference Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
thus thesis by group.unit_left, group.{assoc, inverse_left};  
then thesis by algebra.group.*;
```

Review Point:

- Citation repair remains a small source edit.
- Bulk/grouped forms must be convenient without making dependencies invisible; artifacts record the actually used facts.

Example:

```
then then is by group.un1_left, group.unnest, unverse1_left);
then then is by algebra.group.*;
```

Review Point:

- Citation repair remains a small source edit.
- Bulk/grouped forms must be convenient without making dependencies invisible; artifacts record the actually used facts.

Presenter script: Read aloud:

- Citation repair remains a small source edit.
- Bulk/grouped forms must be convenient without making dependencies invisible; artifacts record the actually used facts.

After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```

registration_block ::= "registration"
                    { registration_content }
                    "end" ";" ;

registration_item ::= existential_registration
                  | conditional_registration
                  | functorial_registration
                  | reduction_registration ;

conditional_registration ::= "cluster" label ":"

```

Grammar sketch:

```

antecedent_adjectives "->"
consequent_adjectives
"for" type_expression ";"
"coherence" justification ";" ;

```

└ Part 2. Language Specification

└ 2.31 - Registrations, Clusters, And Reductions (1/2)

Grammar & kernel:

```

cluster reg_cluster := "reg_cluster"
  { reg_cluster, reg_cluster }
  "end" ;

```

```

cluster reg_cluster := "reg_cluster"
  { reg_cluster, reg_cluster }
  { reg_cluster, reg_cluster }
  { reg_cluster, reg_cluster }

```

```

cluster reg_cluster := "reg_cluster"

```

Grammar & kernel:

```

cluster reg_cluster := "reg_cluster"
  { reg_cluster, reg_cluster }
  { reg_cluster, reg_cluster }
  { reg_cluster, reg_cluster }

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Registrations, Clusters, And Reductions (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
registration
  cluster nonempty_group:
    associative unital -> non empty for Magma;
  coherence by group.nonempty_from_unit;
end;
```

Review Point:

- Registrations remain a first-class Mizar strength.
- The syntax adds labels and traceability so automatic propagation is reviewable and import-filtered.

Example:

```
registrations
  cluster nonempty, group;
  association units; → can empty for Magma;
  coherence by group, nonempty, from_unit;
end;
```

Review Point:

- Registration is a first-class Mizar strength.
- The syntax adds labels and traceability so automatic propagation is reviewable and import-filtered.

Presenter script: Read aloud:

- Registrations remain a first-class Mizar strength.
- The syntax adds labels and traceability so automatic propagation is reviewable and import-filtered.

After the example, say which design obligation the syntax makes visible.

Grammar sketch:

```
reduction_registration ::= "reduce" label ":"  
                        term_expression "to" term_expression ";"  
                        "reducibility" justification ";" ;
```

Example:

```
registration  
  let n be Nat;  
  reduce NatAddZero: n + 0 to n;  
  reducibility by nat.add_zero;  
end;
```


Grammar sketch:

```
reduction_registration ::= "reduce" label ":"  
                        "let" expression "is" term_expression ";"  
                        "reducibility" just if is nil is nil ";" ;
```

Example:

```
registration  
  let n be Nat;  
  reduce NatAddTerm: n + 8 is n;  
  reducibility by nat_add_term;  
end;
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Reduction Registration Semantics (1/2). Point to the visible example, question, or diagram before moving on.

Design Reason:

- A reduction is an oriented simplification rule backed by an equality proof.
- The right-hand side must be strictly smaller under the language's simplification order, so imported rules cannot create rewrite cycles.
- Normalization is deterministic: rule selection is specificity-first, with an FQN tie-break when needed.
- The classic unoriented `identify` idea is not used here; a simplifying equivalence must be written as an auditable `reduce`.

Design Reason:

- A reduction is an oriented simplification rule backed by an equality proof.
- The right-hand side must be strictly smaller under the language's simplification order, so imported rules cannot create rewrite cycles.
- Normalization is deterministic: rule selection is specificity-first, with an FQN tie-break when needed.
- The classic unoriented identify idea is not used here; a simplifying equivalence must be written as an auditable reduce.

Presenter script: Read aloud:

- A reduction is an oriented simplification rule backed by an equality proof.
- The right-hand side must be strictly smaller under the language's simplification order, so imported rules cannot create rewrite cycles.
- Normalization is deterministic: rule selection is specificity-first, with an FQN tie-break when needed.
- The classic unoriented identify idea is not used here; a simplifying equivalence must be written as an auditable reduce.

Grammar sketch:

```
template_definition ::= definition_block ;  
template_parameter_decl ::= definition_parameter_decl ;  
  
let_type ::= "type" [ "extends" bound_type ]  
           | type_expression ;  
  
template_item ::= attr_def | mode_def | struct_def  
               | pred_def | func_def | algorithm_def  
               | theorem_item | registration_item ;
```

Grammar sketch:

```
template_args ::= "[" template_arg { "," template_arg } "]" ;
```

Grammar sketch:

```
template_argument ::= argument_value ;
template_parameter ::= argument_value ;

def_type ::= 'type' | 'record' | 'enum' ;
| type_modifier ;

template ::= { type_or_enum_or_record_or_enum_def | struct_def | class_def | namespace_def |
| struct_def | record_def | enum_def | namespace_def |
| struct_def | record_def | enum_def | namespace_def ;
```

Grammar sketch:

```
template_arg ::= "[" template_arg "," template_arg "]" ;
```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Template Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Example:

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

Review Point:

- Templates are not a cosmetic replacement for scheme.
- They are the common syntax for parameterized definitions, theorem schemas, structures, registrations, and algorithms.
- of and over remain readable shorthands; bracket form is canonical for identity and tooling.

Example:

```
def isPrime
  let T be type;
  assume N is prime T; where
  f is id carrier -> T;
  f is id carrier -> N is prime of T;
end;
```

Review Point:

- Templates are not a cosmetic replacement for schemes.
- They are the common syntax for parameterized definitions, theorem schemas, structures, registrations, and algorithms.
- `if` and `over` remain readable shorthands; bracket form is canonical for identity and tooling.

Presenter script: Read aloud:

- Templates are not a cosmetic replacement for scheme.
- They are the common syntax for parameterized definitions, theorem schemas, structures, registrations, and algorithms.
- `if` and `over` remain readable shorthands; bracket form is canonical for identity and tooling.

After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Grammar sketch:

```

template_args ::= "[" template_arg { "," template_arg } "]" ;
let_constraint ::= "such" "that" formula ;
let_type      ::= "type" [ "extends" bound_type ]
                  | type_expression ;

```

Example:

```

definition
  let F be Field;
  let V be VectorSpace[F] such that V is finite_dimensional;
  mode BasisDef: Basis[V] is finite linearly_independent Subset of V.carrier;
end;

Product[Ring](s)      :: explicit template argument
Product(s)            :: valid only when the argument type determines Ring
M of A, B             :: greedy shorthand for M[A, B] when arity matches

```


└ Part 2. Language Specification

└ 2.34 - Template Constraints And Inference (1/2)

Grammar sketch:

```
template_arg ::= "[" template_arg ( "," template_arg ) "]" ;
val_constraint ::= "var" "is" "distinct" ;
val_type ::= "type" [ "outside" ] kind_type [
    | type_expression ;
```

Example:

```
isDistinct
  let F be Field ;
  let F be RecurSpace1P such that F is distinct_dimensional ;
  mean_dimensional some1P1 is finite_dimensional Subset of F carrier ;
end ;

Product1Sing1al
  :: explicit template argument
Product1al
  :: valid only when the argument type determines Sing
  B of A, B
  :: greedy shorthand for B1A, B1 when arity matches
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Template Constraints And Inference (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Bracket arguments are canonical because they preserve identity for tools.
- `of` and `over` are kept as readable shorthands, but greedy parsing must be deterministic.
- A `such that` constraint is a use-site obligation, not a hidden global assumption.

Review Point:

- Bracket arguments are canonical because they preserve identity for tools.
- of and over are kept as readable shorthands, but greedy parsing must be deterministic.
- A such that constraint is a use-site obligation, not a hidden global assumption.

Presenter script: Read aloud:

- Bracket arguments are canonical because they preserve identity for tools.
- of and over are kept as readable shorthands, but greedy parsing must be deterministic.
- A such that constraint is a use-site obligation, not a hidden global assumption.

Example:

```
@show_type(total + x)

@show_resolution
Product[R](s);

@proof_hint(max_axioms: 32, solver: vampire)
thus result = unit(G) by group.identity_unique;

@latex("\\mathbb{Z}")
func IntSetDef: INT -> set equals IntegerSet;
```

Example:

```
Has_appointment = 1;  
Has_reservation  
ProductID id;  
  
Appt_id, res_id, status: 00, status: required  
case result = success by group, status, unique;  
  
Status(1) %status(20%)  
from: Scheduled: 000 -> not again in the future;
```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Annotation Syntax (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Annotations are source syntax, but they must be semantic-neutral unless a specific proof-development profile treats them as hints for search.
- They guide editors, documentation, proof search, and AI agents.
- They do not become proof evidence merely by being present in the source.
- Fixed diagnostic annotations such as `@show_type(...)` and `@show_resolution` report state; they do not attach logical content to a declaration.

Review Point:

- Annotations are source syntax, but they must be semantic-neutral unless a specific proof-development profile treats them as hints for search.
- They guide editors, documentation, proof search, and AI agents.
- They do not become proof evidence merely by being present in the source.
- Fixed diagnostic annotations such as `@show_type(...)` and `@show_resolution_report state`; they do not attach logical content to a declaration.

Presenter script: Read aloud:

- Annotations are source syntax, but they must be semantic-neutral unless a specific proof-development profile treats them as hints for search.
- They guide editors, documentation, proof search, and AI agents.
- They do not become proof evidence merely by being present in the source.
- Fixed diagnostic annotations such as `@show_type(...)` and `@show_resolution_report state`; they do not attach logical content to a declaration.

Slow down: this is language-specification review, not only implementation detail.

Example:

```
definition
  let x, y be Integer;

  algorithm max2(x, y) -> Integer
    ensures (result = x or result = y) & x <= result & y <= result
  do
    if x <= y do
      return y;
    end;
    return x;
```

Example:

```
    end;
  end;
```


Example:

```
def function m
  let x, y be Integer;
  algorithm m(x, y) → Integer
  result is result = 0 or result = y if x <= result & y <= result
do
  if x <= y then
    result is y;
  else
    result is x;
end
```

Example:

```
end;
end;
```

Presenter script: After the example, say which design obligation the syntax makes visible.

Slow down: this is language-specification review, not only implementation detail.

Use this as the transition: Algorithm Syntax Boundary (1/2). Point to the visible example, question, or diagram before moving on.

Review Point:

- Algorithms are part of the language surface, but they are not allowed to redefine theorem truth.
- They are declared inside `definition` blocks, and the public interface is the signature plus contracts, not the body.
- Contracts, invariants, termination measures, and `by computation` generate verification obligations that pass through the same trusted boundary.

Review Point:

- Algorithms are part of the language surface, but they are not allowed to redefine theorem truth.
- They are declared inside definition blocks, and the public interface is the signature plus contracts, not the body.
- Contracts, invariants, termination measures, and by-computation generate verification obligations that pass through the same trusted boundary.

Presenter script: Read aloud:

- Algorithms are part of the language surface, but they are not allowed to redefine theorem truth.
- They are declared inside definition blocks, and the public interface is the signature plus contracts, not the body.
- Contracts, invariants, termination measures, and by-computation generate verification obligations that pass through the same trusted boundary.

Slow down: this is language-specification review, not only implementation detail.

Questions for Bialystok:

- Which legacy Mizar constructs must keep near-identical syntax for cultural and migration reasons?
- Which constructs may become more explicit because the current implicit form hides dependencies?
- Are `field` / `property` / `attribute` distinctions readable enough in real algebraic examples?
- Is `one-parent-per-inherit` acceptable for multiple-inheritance-heavy MML fragments?
- Do the explicit operator-precedence rules match the expectations of existing Mizar notation users?
- Are `sethood` checks visible enough for Fraenkel-style set formation without making ordinary set notation too heavy?
- Which predicate/functor properties should be migrated first because existing proofs rely on them implicitly?

Questions for Białystok:

- Which legacy Mizar constructs must keep near-identical syntax for cultural and migration reasons?
- Which constructs may become more explicit because the current implicit form hides dependencies?
- Are field / property / attribute distinctions readable enough in real algebraic examples?
- Is one-parent-per-inherit acceptable for multiple-inheritance-heavy MML fragments?
- Do the explicit operator-precedence rules match the expectations of existing Mizar notation users?
- Are authors checking double enough for `FreeStyle` set formation without making a library set notation too heavy?
- Which predicate/function symbols should be migrated first because existing proofs rely on them implicitly?

Presenter script: Read aloud:

- Which legacy Mizar constructs must keep near-identical syntax for cultural and migration reasons?
- Which constructs may become more explicit because the current implicit form hides dependencies?
- Are field / property / attribute distinctions readable enough in real algebraic examples?
- Is one-parent-per-inherit acceptable for multiple-inheritance-heavy MML fragments?
- Do the explicit operator-precedence rules match the expectations of existing Mizar notation users?

Slow down: this is language-specification review, not only implementation detail.

Questions for Bialystok:

- Is oriented reduce a readable replacement for implicit simplification and older identification idioms?
- Are template brackets acceptable as canonical identity syntax if `of` and `over` remain source shorthands?
- Where should template inference stop and require explicit `[T]` arguments or `qua` views?
- Which proof statement forms are missing from this surface review?

Questions for Babelok:

- Is oriented reduce a readable replacement for implicit simplification and older identification idioms?
- Are template brackets acceptable as canonical identity syntax if of and over remain source shorthands?
- Where should template inference stop and require explicit [T] arguments or qua views?
- Which proof statement forms are missing from this surface view?

Presenter script: Read aloud:

- Is oriented reduce a readable replacement for implicit simplification and older identification idioms?
- Are template brackets acceptable as canonical identity syntax if of and over remain source shorthands?
- Where should template inference stop and require explicit [T] arguments or qua views?
- Which proof statement forms are missing from this surface review?

Slow down: this is language-specification review, not only implementation detail.

- Current: `environ` gives each article its vocabulary, notation, constructors, registrations, requirements, and visible theorem base.
- Evo: the same roles must become explicit imports, prelude policy, package metadata, and reproducible dependency reports.
- Review question: which current `environ` roles must remain visually familiar during migration, and which can be replaced by generated reports?

- Current: `environ` gives each article its vocabulary, notation, constructors, registrations, requirements, and visible theorem base.
- Evo: the same roles must become explicit imports, `prelude policy`, package metadata, and reproducible dependency reports.
- Review question: which current `environ` roles must remain visually familiar during migration, and which can be replaced by generated reports?

Presenter script: Read aloud:

- Current: `environ` gives each article its vocabulary, notation, constructors, registrations, requirements, and visible theorem base.
- Evo: the same roles must become explicit imports, `prelude policy`, package metadata, and reproducible dependency reports.
- Review question: which current `environ` roles must remain visually familiar during migration, and which can be replaced by generated reports?

- Current: article acceptance and MML inclusion determine what later articles can use, but the source file has limited public/private interface syntax.
- Evo: export, public, private, opaque imports, and separate compilation define the reusable API surface.
- Review question: which legacy article facts should become public module API, and which should become private proof support?

- Current: article acceptance and MML inclusion determine what later articles can use, but the source file has limited public/private interface syntax.
- Evo: export, public, private, opaque imports, and separate compilation define the reusable API surface.
- Review question: which legacy article facts should become public module API, and which should become private proof support?

Presenter script: Read aloud:

- Current: article acceptance and MML inclusion determine what later articles can use, but the source file has limited public/private interface syntax.
- Evo: export, public, private, opaque imports, and separate compilation define the reusable API surface.
- Review question: which legacy article facts should become public module API, and which should become private proof support?

- Current: theorem and constructor identity is tied to article names, labels, and MML inclusion history.
- Evo: path-derived FQNs and module-qualified labels make identity independent of the local import spelling.
- Review question: what identity metadata is needed to preserve old citations while allowing package/module reorganization?

└ Part 2. Language Specification

└ 2.40 - Detail: Symbol Identity

- Current: theorem and constructor identity is tied to article names, labels, and MML inclusion history.
- Evo: path-derived FQNs and module-qualified labels make identity independent of the local import spelling.
- Review question: what identity metadata is needed to preserve old citations while allowing package/module reorganization?

Presenter script: Read aloud:

- Current: theorem and constructor identity is tied to article names, labels, and MML inclusion history.
- Evo: path-derived FQNs and module-qualified labels make identity independent of the local import spelling.
- Review question: what identity metadata is needed to preserve old citations while allowing package/module reorganization?

- Current: Mizar supports rich mathematical notation, including synonyms and antonyms for alternative phrasing and natural negation.
- Evo: arbitrary operator symbols are concentrated in `func` and `pred`, while `mode`, `attribute`, and structure names remain identifier-like; synonym and antonym equivalence is preserved.
- Review question: which legacy symbolic constructors need compatibility spelling, and which should migrate to clearer constructor names?

- Current: Mizar supports rich mathematical notation, including synonyms and antonyms for alternative phrasing and natural negation.
- Evo: arbitrary operator symbols are concentrated in func and pred, while mode, attribute, and structure names remain identifier-like; synonym and antonym equivalence is preserved.
- Review question: which legacy symbolic constructors need compatibility spelling, and which should migrate to clearer constructor names?

Presenter script: Read aloud:

- Current: Mizar supports rich mathematical notation, including synonyms and antonyms for alternative phrasing and natural negation.
- Evo: arbitrary operator symbols are concentrated in func and pred, while mode, attribute, and structure names remain identifier-like; synonym and antonym equivalence is preserved.
- Review question: which legacy symbolic constructors need compatibility spelling, and which should migrate to clearer constructor names?

- Current: an article environment determines which symbols and notations are available, with many effects hidden from local text.
- Evo: the import prelude is pre-scanned, the active lexicon is source-position dependent, and declarations become active only after their item is complete.
- Review question: where will migration need generated diagnostics for no-forward-reference or dot/operator disambiguation changes?

- Current: an article environment determines which symbols and notations are available, with many effects hidden from local text.
- Evo: the import prelude is pre-scanned, the active lexicon is source-position dependent, and declarations become active only after their item is complete.
- Review question: where will migration need generated diagnostics for no-forward-reference or dot/operator disambiguation changes?

Presenter script: Read aloud:

- Current: an article environment determines which symbols and notations are available, with many effects hidden from local text.
- Evo: the import prelude is pre-scanned, the active lexicon is source-position dependent, and declarations become active only after their item is complete.
- Review question: where will migration need generated diagnostics for no-forward-reference or dot/operator disambiguation changes?

- Current: Mizar's soft type system is a central identity feature, layered over set-theoretic objects.
- Evo: soft typing is preserved while `object/set`, `radix` and `mode heads`, `type erasure`, `widening/narrowing`, and `reconsider obligations` are exposed.
- Review question: which type obligations should be visible in source, and which should remain verifier metadata?

- Current: Mizar's soft type system is a central identity feature, layered over set-theoretic objects.
- Evo: soft typing is preserved while object/set, radix and mode heads, type erasure, widening/narrowing, and reconsider obligations are exposed.
- Review question: which type obligations should be visible in source, and which should remain verifier metadata?

Presenter script: Read aloud:

- Current: Mizar's soft type system is a central identity feature, layered over set-theoretic objects.
- Evo: soft typing is preserved while object/set, radix and mode heads, type erasure, widening/narrowing, and reconsider obligations are exposed.
- Review question: which type obligations should be visible in source, and which should remain verifier metadata?

- Current: structure declarations combine parent structure, fields, selectors, and constructor layout in compact legacy syntax.
- Evo: `struct`, `field`, `property`, and `inherit` separate layout, canonical derived data, and inherited views.
- Review question: when migrating a legacy selector, is it intrinsic structure data, a derived property, or an inherited view?

- Current: structure declarations combine parent structure, fields, selectors, and constructor in layout in compact legacy syntax.
- Evo: struct, field, property, and inherit separate layout, canonical derived data, and inherited views.
- Review question: when migrating a legacy selector, is it intrinsic structure data, a derived property or an inherited view?

Presenter script: Read aloud:

- Current: structure declarations combine parent structure, fields, selectors, and constructor layout in compact legacy syntax.
- Evo: struct, field, property, and inherit separate layout, canonical derived data, and inherited views.
- Review question: when migrating a legacy selector, is it intrinsic structure data, a derived property, or an inherited view?

- Current: inheritance is powerful but inherited field mapping is often read through structure syntax and naming convention.
- Evo: each `inherit` statement records one parent relation, including mapping, renaming, narrowing, and coherence evidence.
- Review question: which algebraic examples best expose diamond inheritance and coherence obligations without overwhelming the audience?

- Current: inheritance is powerful but inherited field mapping is often read through structure syntax and naming convention.
- Evo: each inherit statement records one parent relation, including mapping, renaming, narrowing, and coherence evidence.
- Review question: which algebraic examples best expose diamond inheritance and coherence obligations without overwhelming the audience?

Presenter script: Read aloud:

- Current: inheritance is powerful but inherited field mapping is often read through structure syntax and naming convention.
- Evo: each inherit statement records one parent relation, including mapping, renaming, narrowing, and coherence evidence.
- Review question: which algebraic examples best expose diamond inheritance and coherence obligations without overwhelming the audience?

- Current: modes and adjectives make Mizar text close to mathematical prose.
- Evo: modes remain type abbreviations/refinements, while attributes are explicit type-refining predicates used by clustering.
- Review question: which current adjectives are cultural vocabulary that should be preserved exactly?

└ Part 2. Language Specification

└ 2.46 - Detail: Modes And Attributes

- Current: modes and adjectives make Mizar text close to mathematical prose.
- Evo: modes remain type abbreviations/refinements, while attributes are explicit type-refining predicates used by clustering.
- Review question: which current adjectives are cultural vocabulary that should be preserved exactly?

Presenter script: Read aloud:

- Current: modes and adjectives make Mizar text close to mathematical prose.
- Evo: modes remain type abbreviations/refinements, while attributes are explicit type-refining predicates used by clustering.
- Review question: which current adjectives are cultural vocabulary that should be preserved exactly?

- Current: `func` and `pred` definitions use familiar `equals` and `means` styles with correctness obligations.
- Evo: these styles remain, but existence, uniqueness, coherence, compatibility, and assume side conditions become explicit obligations and artifact entries.
- Review question: which correctness obligations should be source-visible in a migration diff?

- Current: func and pred definitions use familiar equals and means styles with correctness obligations.
- Evo: these styles remain, but existence, uniqueness, coherence, compatibility and assume side conditions become explicit obligations and artifact entries.
- Review question: which correctness obligations should be source-visible in a migration diff?

Presenter script: Read aloud:

- Current: func and pred definitions use familiar equals and means styles with correctness obligations.
- Evo: these styles remain, but existence, uniqueness, coherence, compatibility, and assume side conditions become explicit obligations and artifact entries.
- Review question: which correctness obligations should be source-visible in a migration diff?

- Current: overloaded symbols and redefinitions support compact mathematical notation over many related types.
- Evo: ordinary overload roots, same-root `redefine` families, coherence with, and refinement joins are separated and recorded.
- Review question: how should tools explain a changed overload choice to a user who only sees familiar mathematical notation?

Presenter script: Read aloud:

- Current: overloaded symbols and redefinitions support compact mathematical notation over many related types.
- Evo: ordinary overload roots, same-root redefine families, coherence with, and refinement joins are separated and recorded.
- Review question: how should tools explain a changed overload choice to a user who only sees familiar mathematical notation?

- Current: registrations and clusters are one of Mizar's major automation strengths, but their local effect can be hard to inspect.
- Evo: labeled registration items, import-filtered cluster graphs, reduction indexes, and replayable traces make propagation and rewriting explainable.
- Review question: which cluster explanations are essential for trusting a migrated article?

- Current: registrations and clusters are one of Mizar's major automation strengths, but their local effect can be hard to inspect.
- Evo: labeled registration items, import-filtered cluster graphs, reduction indexes, and replayable traces make propagation and rewriting explainable.
- Review question: which cluster explanations are essential for trusting a migrated article?

Presenter script: Read aloud:

- Current: registrations and clusters are one of Mizar's major automation strengths, but their local effect can be hard to inspect.
- Evo: labeled registration items, import-filtered cluster graphs, reduction indexes, and replayable traces make propagation and rewriting explainable.
- Review question: which cluster explanations are essential for trusting a migrated article?

- Current: classical scheme and of/over parameterization cover many reusable reasoning patterns.
- Evo: first-class templates unify parameterized definitions, structures, attributes, functors, predicates, algorithms, and theorem schemas.
- Review question: which existing schemes should be first migration targets for template-style presentation?

- Current: classical scheme and of/over parameterization cover many reusable reasoning patterns.
- Evo: first-class templates unify parameterized definitions, structures, attributes, functors, predicates, algorithms, and theorem schemas.
- Review question: which existing schemas should be first migration targets for template-style presentation?

Presenter script: Read aloud:

- Current: classical scheme and of/over parameterization cover many reusable reasoning patterns.
- Evo: first-class templates unify parameterized definitions, structures, attributes, functors, predicates, algorithms, and theorem schemas.
- Review question: which existing schemas should be first migration targets for template-style presentation?

- Current: Jaśkowski-style proof text with `let`, `assume`, `thus`, `hence`, `thesis`, and diffuse reasoning is central to Mizar readability.
- Evo: the readable skeleton is preserved while extracted obligations, thesis states, and source spans are emitted as metadata.
- Review question: how much proof-state metadata helps users without making the proof look tactic-driven?

- Current: Jaśkowski-style proof text with let, assume, thus, hence, thesis, and diffuse reasoning is central to Mizar readability.
- Evo: the readable skeleton is preserved while extracted obligations, thesis states, and source spans are emitted as metadata.
- Review question: how much proof-state metadata helps users without making the proof look tactic-driven?

Presenter script: Read aloud:

- Current: Jaśkowski-style proof text with let, assume, thus, hence, thesis, and diffuse reasoning is central to Mizar readability.
- Evo: the readable skeleton is preserved while extracted obligations, thesis states, and source spans are emitted as metadata.
- Review question: how much proof-state metadata helps users without making the proof look tactic-driven?

- Current: the normal publication unit is a verified theorem item.
- Evo: open, assumed, and conditional distinguish unsettled propositions, deliberate assumptions, and results depending on non-clean material.
- Review question: what policy is acceptable for non-clean items in published or package-distributed material?

- Current: the normal publication unit is a verified theorem item.
- Evo: open, assumed, and conditional distinguish unsettled propositions, deliberate assumptions, and results depending on non-clean material.
- Review question: what policy is acceptable for non-clean items published in package-distributed material?

Presenter script: Read aloud:

- Current: the normal publication unit is a verified theorem item.
- Evo: open, assumed, and conditional distinguish unsettled propositions, deliberate assumptions, and results depending on non-clean material.
- Review question: what policy is acceptable for non-clean items published or package-distributed material?

- Current: by citations name visible facts and keep proof steps compact.
- Evo: grouped citations, bulk citations, used-axiom recording, and citation refinement support small verifier-checked repairs.
- Review question: when should a migration tool minimize citations, and when should it preserve the author's original citation style?

- Current: by citations name visible facts and keep proof steps compact.
- Evo: grouped citations, bulk citations, used-axiom recording, and citation refinement support small verifier-checked repairs.
- Review question: when should a migration tool minimize citations, and when should it preserve the author's original citation style?

Presenter script: Read aloud:

- Current: by citations name visible facts and keep proof steps compact.
- Evo: grouped citations, bulk citations, used-axiom recording, and citation refinement support small verifier-checked repairs.
- Review question: when should a migration tool minimize citations, and when should it preserve the author's original citation style?

- Current: qua, widening, narrowing, and local disambiguation help users select intended type views.
- Evo: source-level qua is preserved, while inserted coercions, selected views, and overload metadata become inspectable artifacts.
- Review question: which implicit choices should be shown in source during migration, and which should remain hover/explanation data?

- Current: qua, widening, narrowing, and local disambiguation help users select intended type views.
- Evo: source-level qua is preserved, while inserted coercions, selected views, and overloaded metadata become inspectable artifacts.
- Review question: which implicit choices should be shown in source during migration, and which should remain hover/explanation data?

Presenter script: Read aloud:

- Current: qua, widening, narrowing, and local disambiguation help users select intended type views.
- Evo: source-level qua is preserved, while inserted coercions, selected views, and overload metadata become inspectable artifacts.
- Review question: which implicit choices should be shown in source during migration, and which should remain hover/explanation data?

- Current: Mizar is primarily a mathematical proof language, not a general verified programming surface.
- Evo: algorithm, contracts, invariants, termination measures, MVM execution, and by computation connect verified computation to proof.
- Review question: which computational examples would demonstrate value without distracting from the mathematical proof language?

- Current: Mizar is primarily a mathematical proof language, not a general verified programming surface.
- Evo: algorithm, contracts, invariants, termination measures, MVM execution, and by computation connect verified computation to proof.
- Review question: which computational examples would demonstrate value without distracting from the mathematical proof language?

Presenter script: Read aloud:

- Current: Mizar is primarily a mathematical proof language, not a general verified programming surface.
- Evo: algorithm, contracts, invariants, termination measures, MVM execution, and by computation connect verified computation to proof.
- Review question: which computational examples would demonstrate value without distracting from the mathematical proof language?

- Current: comments and informal tool conventions can help readers but do not form a stable verifier-facing interface.
- Evo: semantic-neutral annotations guide proof search, display inferred state, and render notation without changing theorem truth.
- Review question: which annotations are useful enough to standardize, and which should stay in external tooling?

- Current: comments and informal tool conventions can help readers but do not form a stable verifier-facing interface.
- Evo: semantic-neutral annotations guide proof search, display inferred state, and render notation without changing theorem truth.
- Review question: which annotations are useful enough to standardize, and which should stay in external tooling?

Presenter script: Read aloud:

- Current: comments and informal tool conventions can help readers but do not form a stable verifier-facing interface.
- Evo: semantic-neutral annotations guide proof search, display inferred state, and render notation without changing theorem truth.
- Review question: which annotations are useful enough to standardize, and which should stay in external tooling?

- Current: verifier output is primarily human-facing text.
- Evo: stable diagnostic codes, primary and secondary spans, fix suggestions, lazy explanation artifacts, and LSP records make errors actionable.
- Review question: which diagnostic explanations would most reduce migration friction for current MML maintainers?

- Current: verifier output is primarily human-facing text.
- Evo: stable diagnostic codes, primary and secondary spans, fix suggestions, lazy explanation artifacts, and LSP records make errors actionable.
- Review question: which diagnostic explanations would most reduce migration friction for current MML maintainers?

Presenter script: Read aloud:

- Current: verifier output is primarily human-facing text.
- Evo: stable diagnostic codes, primary and secondary spans, fix suggestions, lazy explanation artifacts, and LSP records make errors actionable.
- Review question: which diagnostic explanations would most reduce migration friction for current MML maintainers?

- Current: proof automation is valuable, but the trusted boundary is not presented as a portable artifact protocol.
- Evo: ATPs search, certificate artifacts record evidence, and the kernel independently accepts or rejects that evidence.
- Review question: which certificate failures should be user-facing, and which are implementation/debugging details?

- Current: proof automation is valuable, but the trusted boundary is not presented as a portable artifact protocol.
- Evo: ATPs search, certificate artifacts record evidence, and the kernel independently accepts or rejects that evidence.
- Review question: which certificate failures should be user-facing, and which are implementation/debugging details?

Presenter script: Read aloud:

- Current: proof automation is valuable, but the trusted boundary is not presented as a portable artifact protocol.
- Evo: ATPs search, certificate artifacts record evidence, and the kernel independently accepts or rejects that evidence.
- Review question: which certificate failures should be user-facing, and which are implementation/debugging details?

- Current: MML is organized around articles and library inclusion.
- Evo: `mizar.pkg`, lockfiles, SemVer, features, compatibility checks, and cached artifacts support reproducible package-oriented development.
- Review question: where should the standard library, published articles, third-party packages, and local experiments sit in the namespace policy?

- Current: MML is organized around articles and library inclusion.
- Evo: mizar.pkg, lockfiles, SemVer, features, compatibility checks, and cached artifacts support reproducible package-oriented development.
- Review question: where should the standard library, published articles, third-party packages, and local experiments sit in the namespace policy?

Presenter script: Read aloud:

- Current: MML is organized around articles and library inclusion.
- Evo: mizar.pkg, lockfiles, SemVer, features, compatibility checks, and cached artifacts support reproducible package-oriented development.
- Review question: where should the standard library, published articles, third-party packages, and local experiments sit in the namespace policy?

- Current: accepted article output is the main reuse boundary.
- Evo: dependency slices, VC anchors, witness hashes, and cache keys allow reuse, but cache reuse is never proof authority.
- Review question: what clean-build equivalence tests would make incremental verification trustworthy?

- Current: accepted article output is the main reuse boundary.
- Evo: dependency slices, VC anchors, witness hashes, and cache keys allow reuse, but cache reuse is never proof authority.
- Review question: what clean-build equivalence tests would make incremental verification trustworthy?

Presenter script: Read aloud:

- Current: accepted article output is the main reuse boundary.
- Evo: dependency slices, VC anchors, witness hashes, and cache keys allow reuse, but cache reuse is never proof authority.
- Review question: what clean-build equivalence tests would make incremental verification trustworthy?

- Current: source comments, MML browsing, and Formalized Mathematics pages serve related but distinct reading needs.
- Evo: `mizar doc` uses verified artifacts, labels, FQNs, documentation comments, and `@latex` annotations to produce API documentation.
- Review question: which documentation warnings should be optional lint, and which should block publication profiles?

Mizar Evo

└ Part 2. Language Specification

└ 2.61 - Detail: Documentation Generation

Presenter script: Read aloud:

- Current: source comments, MML browsing, and Formalized Mathematics pages serve related but distinct reading needs.
- Evo: mizar doc uses verified artifacts, labels, FQNs, documentation comments, and @latex annotations to produce API documentation.
- Review question: which documentation warnings should be optional lint, and which should block publication profiles?

- Current: there is no general source-level workflow for extracting verified executable code from Mizar articles.
- Evo: terminating algorithms, ghost erasure, target-neutral runtime IR, and extractor configuration make executable output a downstream artifact.
- Review question: which target languages and mathematical domains are the right first extraction benchmarks?

- Current: there is no general source-level workflow for extracting verified executable code from Mizar articles.
- Evo: terminating algorithms, ghost erasure, target-neutral runtime IR, and extractor configuration make executable output a downstream artifact.
- Review question: which target languages and mathematical domains are the right first extraction benchmarks?

Presenter script: Read aloud:

- Current: there is no general source-level workflow for extracting verified executable code from Mizar articles.
- Evo: terminating algorithms, ghost erasure, target-neutral runtime IR, and extractor configuration make executable output a downstream artifact.
- Review question: which target languages and mathematical domains are the right first extraction benchmarks?

- Current: article identity, publication exposition, and library reuse are closely connected.
- Evo: publication metadata links articles to reusable library modules, package versions, and stable theorem identities.
- Review question: how can Evo preserve citation value while letting the reusable library evolve independently?

└ Part 2. Language Specification

└ 2.63 - Detail: Formalized Mathematics Link

- Current: article identity, publication exposition, and library reuse are closely connected.
- Evo: publication metadata links articles to reusable library modules, package versions, and stable theorem identities.
- Review question: how can Evo preserve citation value while letting the reusable library evolve independently?

Presenter script: Read aloud:

- Current: article identity, publication exposition, and library reuse are closely connected.
- Evo: publication metadata links articles to reusable library modules, package versions, and stable theorem identities.
- Review question: how can Evo preserve citation value while letting the reusable library evolve independently?

Exact current MML excerpt:

```
environ
```

```
vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,  
             FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;  
notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,  
          CARD_1, STRUCT_0;  
constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;  
registrations ZFMISC_1, CARD_1, STRUCT_0;  
theorems STRUCT_0;
```

Exact current MML excerpt:

```
begin :: Additive structures
```

Exact current MML excerpt:

[illegible]

Exact current MML excerpt:

begin :: Additive structure

Presenter script: Say:

- Source: current MML algstr_0.miz, lines 15-25.
- The point is not that `environ` is bad. The point is that dependency classes are article-level and need translation into explicit module/package data.

```
import std.algebra.structure.sorted;  
import std.function.binop;  
import std.algebra.structure.magma;
```

Main differences:

- file-scoped import prelude;
- package/module namespace;
- import closure as a build-system input;
- stable FQNs derived from path and module.

Design Reason:

- The prelude is deliberately file-scoped so lexing, disambiguation, and AI context extraction can build one stable imported base environment before reading the body.
- This makes dependency reports reproducible: a tool can say which import provided each symbol, notation, registration, and theorem.

Mizar Evo

└ Part 2. Language Specification

└ 2.65 - Mizar Evo Import Prelude

```
import at d, no go here, at must use a, sort d;  
import at d, must use a, sort d;  
import at d, no go here, at must use a, sort d;
```

Main differences:

- file-scoped import prelude;
- package/module namespace;
- import closure as a build-system input;
- static FQN is derived from path and module.

Design Reason:

- The prelude is deliberately file-scoped so loading, disambiguation, and all context extraction can build on static imported names and names at before reading the body.
- This makes the process reproducible: a tool can say which import provided each symbol, mistake, registration, and theorem.

Presenter script: Read aloud:

- file-scoped import prelude;
- package/module namespace;
- import closure as a build-system input;

After the example, say which design obligation the syntax makes visible.

2.66 - Migration Map: Environment To Imports

Current Mizar role	Evo target
vocabularies	exported symbols and lexical metadata
notations	imported notation metadata
constructors	visible definitions and constructors
registrations	import-scoped registration index
requirements	package or prelude policy
article labels	module-qualified theorem identities

2.66 - Migration Map: Environment To Imports

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Migration Map: Environment To Imports. Point to the visible example, question, or diagram before moving on.

2.56 - Migration Map: Environment To Imports	
Current: Myat rak	Env range
incubation	upright symbol and local mirror
nesting	upright symbol and mirror
regulation	inverted symbol and core action
regulation	upright symbol and core action
article look	package or package path
	inverted symbol and mirror

Key Points:

- Current environments mix several dependency roles.
- Some dependencies are semantic, some syntactic, some automation-facing.
- Migration needs reports explaining what each imported module contributes.
- The purpose of the new boundary is not cosmetic modernization. It is to make old implicit roles reviewable before they become package and artifact dependencies.

Key Points:

- Current environments mix several dependency roles.
- Some dependencies are semantic, some syntactic, some automation-facing.
- Migration needs reports explaining what each imported module contributes.
- The purpose of the new boundary is not cosmetic modernization. It is to make all implicit roles reviewable before they become package and artifact dependencies.

Presenter script: Read aloud:

- Current environments mix several dependency roles.
- Some dependencies are semantic, some syntactic, some automation-facing.
- Migration needs reports explaining what each imported module contributes.
- The purpose of the new boundary is not cosmetic modernization. It is to make old implicit roles reviewable before they become package and artifact dependencies.

Current plain-text ALGSTR_0 exposes the shape:

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

```
definition
  exact {Succed: addRagm (R carrier -> set, add -> kindy of the carrier
  R);
end;
```

Presenter script: Say:

- Source: current MML algstr_0.miz, lines 37-40.
- This is a short exact excerpt; the Beamer slide can use the same snippet with attribution in speaker notes.

Read aloud:

- Current plain-text ALGSTR_0 exposes the shape:

```
definition
```

```
  struct AddMagma where  
    field carrier -> set;  
    field add -> BinOp of carrier;  
end;
```

```
  struct AddLoopStr where  
    field carrier -> non empty set;  
    field add -> BinOp of carrier;  
    property zero -> Element of carrier;
```

```
end;
```

```
  inherit AddMagma extends 1-sorted;  
end;
```

```
definition B
  structure AddMagnum name
    field carrier -> nat;
    field add -> BinOp of carrier;
  end;

  structure AddMagnum name
    field carrier -> non empty nat;
    field add -> BinOp of carrier;
    property zero -> element of carrier;
  end;

  order B: AddMagnum extends B=order;
end;
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Evo Structure Example (1/2). Point to the visible example, question, or diagram before moving on.

Claim:

- Evo makes field declarations and inheritance obligations explicit, while keeping the concept close to Mizar structures.
- Evo also separates `field` from `property`: fields are independent structure components, while properties are uniquely determined mathematical data or obligations associated with the structure.

Design Reason:

- Structure inheritance remains useful only if users can see which fields are shared, renamed, or required by a parent structure.
- Making those obligations explicit turns a migration risk into something the verifier, LSP, and reviewers can discuss.
- Separating fields from properties prevents constructor layout, selector identity, and mathematical laws from being treated as the same kind of dependency.

Claim:

- Evo makes field declarations and inheritance obligations explicit, while keeping the concept close to Mizar structures.
- Evo also separates field from property: fields are independent structure components, while properties are uniquely determined mathematical data or obligations associated with the structure.

Design Reason:

- Structure inheritance remains useful only if users can see which fields are shared, renamed, or required by a parent structure.
- Making those obligations explicit turns a migration risk into something the verifier, LSP, and reviewers can discuss.
- Separating fields from properties prevents constructor layout, selector identity, and mathematical laws from being treated as the same kind of dependency.

Presenter script: Read aloud:

- Evo makes field declarations and inheritance obligations explicit, while keeping the concept close to Mizar structures.
- Evo also separates field from property: fields are independent structure components, while properties are uniquely determined mathematical data or obligations associated with the structure.
- Structure inheritance remains useful only if users can see which fields are shared, renamed, or required by a parent structure.
- Making those obligations explicit turns a migration risk into something the verifier, LSP, and reviewers can discuss.
- Separating fields from properties prevents constructor layout, selector identity, and mathematical laws from being treated as the same kind of dependency.

Design distinction:

Concept	What it means	Why it is separate
field	intrinsic component supplied by the structure value	affects constructor shape, selector identity, extensional equality, and storage-like artifacts
property	uniquely determined value or obligation attached to the structure	affects semantic obligations, coherence, inheritance constraints, and proof/artifact fingerprints
attribute	predicate-style refinement such as <code>empty</code> , <code>trivial</code> , or <code>add-cancelable</code>	participates in registration and cluster propagation rather than structure layout

└ Part 2. Language Specification

└ 2.70 - Why Separate Field And Property? (1/2)

Design distinction:		
Category	Field	Property
Field	Values are supplied by the user or value	Values are supplied by the user or value
Property	Values are supplied by the user or value	Values are supplied by the user or value
Attribute	Values are supplied by the user or value	Values are supplied by the user or value

Presenter script: Say:

- Source design vocabulary: doc/spec/en/05.structures.md distinguishes field from property; doc/spec/en/06.attributes.md defines attributes as type-refining predicates used by clustering.

For the table, read the row labels first, then the design reason.

Message:

- A carrier or operation is a field because changing it changes the object.
- A zero, unit, degree, or similar canonical value may be a property when it is determined by the mathematical structure and needs coherence evidence.
- An attribute is neither stored data nor a selector; it is a reusable logical refinement that automation can propagate.
- This split gives migration tools a better question to ask: "is this legacy item part of the structure's data, a canonical value with proof obligations, or a propagated predicate?"

Message:

- A carrier or operation is a field because changing it changes the object.
- A zero, unit, degree, or similar canonical value may be a property when it is determined by the mathematical structure and needs coherence evidence.
- An attribute is neither stored data nor a selector; it is a reusable logical refinement that automations can propagate.
- This split gives migration tools a better question to ask: "is this legacy item part of the structure's data, a canonical value with proof obligations, or a propagated predicate?"

Presenter script: Read aloud:

- A carrier or operation is a field because changing it changes the object.
- A zero, unit, degree, or similar canonical value may be a property when it is determined by the mathematical structure and needs coherence evidence.
- An attribute is neither stored data nor a selector; it is a reusable logical refinement that automation can propagate.
- This split gives migration tools a better question to ask: "is this legacy item part of the structure's data, a canonical value with proof obligations, or a propagated predicate?"

Design distinction:

Declaration	What it owns	Why it is separate
<code>struct</code>	the local type constructor, fields, and properties	defines the object's own layout and selectors
<code>inherit</code>	the relation to one parent structure	records field/property mapping, renaming, narrowing, and coherence evidence

Message:

- A structure declaration should answer "what is this object made of?"
- An inheritance declaration should answer "how is this object viewed as that parent?"
- Keeping them separate makes multiple inheritance explicit: each parent gets one auditable `inherit` statement instead of one large implicit merge.
- Diamond inheritance then becomes a diagnostic problem with source spans and coherence obligations, not a hidden side effect of declaration order.
- It also gives artifacts cleaner fingerprints: changing a field layout is not the same event as changing an inherited view or its coherence proof.

Part 2. Language Specification

2.71 - Why Separate struct And inherit?

Design distinction:

Declaration	What it means	What it is separate
struct	the local type construction, fields, and properties	gets on the object's own type, and subtypes
inherit	the relation to one parent structure	inherits the program mapping, semantics, meaning, and coherence with its

Message:

- A structure declaration should answer "what is this object made of?"
- An inheritance declaration should answer "how is this object viewed as that parent?"
- Keeping them separate makes multiple inheritance explicit: each parent gets one auditable inherit statement instead of one large implicit merge.
- The need to inherit then becomes a diagnostic problem with source spans and coherence obligations, not a hidden side effect of declaration order.
- It also gives a clearer design perspective: changing a field layout is not the same event as changing a s inherited view on its coherence proof.

Presenter script: Say:

- Source design vocabulary: `doc/spec/en/05.structures.md` requires one parent per inherit statement and uses explicit where blocks for renaming, narrowing, and coherence.

Read aloud:

- A structure declaration should answer "what is this object made of?"
- An inheritance declaration should answer "how is this object viewed as that parent?"
- Keeping them separate makes multiple inheritance explicit: each parent gets one auditable inherit statement instead of one large implicit merge.

For the table, read the row labels first, then the design reason.

```
definition
  struct Magma where
    field carrier -> set;
    field binop -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;
  end;

end;
```

Purpose:

- Support library organization where additive and multiplicative views share structure without relying on hidden informal naming conventions.
- Preserve the mathematical habit of reusing the same carrier through different views, while recording the view translation as checked source.


```

definition
  struct: rename struct
  field carrier -> set;
  field hmap -> kind of carrier;
end;

infix: addFrom struct: rename struct
  field carrier from carrier;
  field add from hmap;
end;
end;

```

Purpose:

- Support library organization where additive and multiplicative views share structure without relying on hidden informal naming conventions.
- Preserve the mathematical habit of reusing the same carrier through different views, while recording the view translation as checked source.

Presenter script: Read aloud:

- Support library organization where additive and multiplicative views share structure without relying on hidden informal naming conventions.
- Preserve the mathematical habit of reusing the same carrier through different views, while recording the view translation as checked source.

After the example, say which design obligation the syntax makes visible.

```
inherit DoubleLoopStr extends AddLoopStr;  
inherit DoubleLoopStr extends MulLoopStr;
```

Message:

- The system should explain when two inheritance paths agree, conflict, or need a proof of coherence.
- This explanation should be available to users, LSP, and AI agents.
- The design goal is to keep multiple-view algebra usable without letting hidden inheritance order decide semantics.

Message:

- The system should explain when two inheritance paths agree, conflict, or need a proof of coherence.
- This explanation should be available to users, LSP, and AI agents.
- The design goal is to keep multiple-view algebra usable without letting hidden inheritance order decide semantics.

Presenter script: Read aloud:

- The system should explain when two inheritance paths agree, conflict, or need a proof of coherence.
- This explanation should be available to users, LSP, and AI agents.
- The design goal is to keep multiple-view algebra usable without letting hidden inheritance order decide semantics.

After the example, say which design obligation the syntax makes visible.

Exact current MML excerpt:

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
Element
  of M;
  coherence;
end;
```

Evo interpretation:

- registration is still a mathematical mechanism;
- the implementation stores an import-filtered, explainable registration graph;
- every automatic attribute propagation should be traceable.

Exact current MML excerpt:

```

registration
  let N be add@magm;
  cluster = right_half-ance table left_half-ance table -> add-ance table for
  N in magm;
  coherence;
end;

```

Evo interpretation:

- registration is still a mathematical mechanism;
- the implementation stores an import-filtered, explainable registration graph;
- every automatic attribute propagation should be traceable.

Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 104-109.
- The excerpt is intentionally short: it shows attribute propagation without making the slide depend on a long registration proof.

Read aloud:

- registration is still a mathematical mechanism;
- the implementation stores an import-filtered, explainable registration graph;
- every automatic attribute propagation should be traceable.

Design Reason:

- Registrations and clusters are one of Mizar's strengths, so Evo should not remove them.
- The change is that automatic propagation must leave a trace that can be audited, cached, and shown to an AI agent without dumping the whole library.

Design Reason:

- Registrations and clusters are one of Mizar's strengths, so Evo should not remove them.
- The change is that automatic propagation must leave a trace that can be audited, cached, and shown to an AI agent without dumping the whole library.

Presenter script: Read aloud:

- Registrations and clusters are one of Mizar's strengths, so Evo should not remove them.
- The change is that automatic propagation must leave a trace that can be audited, cached, and shown to an AI agent without dumping the whole library.

```
consider p be Product(R qua MulStr) such that  
...
```

Message:

- Explicit disambiguation is not a failure of inference.
- It is a readable record of the intended mathematical view.
- It gives AI agents a small safe repair target.


```
consider p be Product (R qua Multi) such that  
...
```

Message:

- Explicit disambiguation is not a failure of inference.
- It is a readable record of the intended mathematical view.
- It gives AI agents a small safe repair target.

Presenter script: Read aloud:

- Explicit disambiguation is not a failure of inference.
- It is a readable record of the intended mathematical view.
- It gives AI agents a small safe repair target.

After the example, say which design obligation the syntax makes visible.

Before:

```
thus thesis by A;
```

After:

```
thus thesis by A, B, C;
```

Exact current MML proof-citation example:

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

Before:

`{base thm is by A;`

After:

`{base thm is by A, B, C;`

Exact current MML proof-citation example:

```
theorem
  for P being non degenerated ZeroMatrix holds 1.P in Headers P
proof
  let P be non degenerated ZeroMatrix;
  not 1.P in (H.P) by TARSKI:def 1;
  hence thesis by XIBLL_1:def 1;
end;
```

Presenter script: Say:

- Source: current MML struct_0.miz, lines 637-643.
- Use this to explain citation repair as a bounded local edit: an agent may propose a missing or more precise citation, but the verifier and kernel evidence decide acceptance.

Message:

- This is a Green AI edit: source-local, verifier-checked, meaning-preserving if accepted with respect to the theorem statement.
- A later refinement pass can minimize citations based on used axioms recorded in artifacts.
- The added dependency still matters; artifacts should record it so a later mizar refine-style pass can reduce noise.

Message:

- This is a Green AI edit: source-local, verifier-checked, meaning-preserving if accepted with respect to the theorem statement.
- A later refinement pass can minimize citations based on used axioms recorded in artifacts.
- The added dependency still matters; artifacts should record it so a later mizar refine-style pass can reduce noise.

Presenter script: Read aloud:

- This is a Green AI edit: source-local, verifier-checked, meaning-preserving if accepted with respect to the theorem statement.
- A later refinement pass can minimize citations based on used axioms recorded in artifacts.
- The added dependency still matters; artifacts should record it so a later mizar refine-style pass can reduce noise.

Bad AI repair:

```
theorem  
  for x be Nat holds  $x + 0 = x$  or  $x = 0$ ;
```

Message:

- Weakening the theorem statement is a Red edit.
- It may make the proof easier but destroys the intended result.
- Ordinary AI agents must not be authorized to apply it; at most they may flag that such a change would require explicit human unsafe-edit review.

Bad AI repair:

```
theorem
  for x be Nat holds x + 0 = x or x = 0;
```

Message:

- Weakening the theorem statement is a Red edit.
- It may make the proof easier but destroys the intended result.
- Ordinary AI agents must not be authorized to apply it; at most they may flag that such a change would require explicit human unsafe-edit review.

Presenter script: Read aloud:

- Weakening the theorem statement is a Red edit.
- It may make the proof easier but destroys the intended result.
- Ordinary AI agents must not be authorized to apply it; at most they may flag that such a change would require explicit human unsafe-edit review.

After the example, say which design obligation the syntax makes visible.

```
@show_type(total + x)

@show_resolution
Product[R](s);

@proof_hint(max_axioms: 32, solver: vampire)
thus result = unit(G) by group.identity_unique;
```

Message:

- Annotations are structured context for humans, tools, and AI.
- Informational annotations should not change logical meaning.
- Hints that influence search must still be treated as proof-development metadata, not as accepted evidence.

Part 2. Language Specification

└ 2.78 - Annotations

```
2.78 - AnsoTools

@show_type(typeof = x)

@show_result in
  Foo{data::Int64};

@read_hint [max_extensions: 33, solver: vcsolve]
  base result = unit{G} by group_id if y_inquire;
```

Message:

- Annotations are not used in code for humans, tools, and AI.
- Informational annotations show AI not change logical meaning.
- Hints that inform a search must **still** be treated as `unsafe` by the meta-data, not as a declared `safe`.

Presenter script: Read aloud:

- Annotations are structured context for humans, tools, and AI.
- Informational annotations should not change logical meaning.
- Hints that influence search must still be treated as proof-development metadata, not as accepted evidence.

After the example, say which design obligation the syntax makes visible.

Proposed Evo sketch:

```

definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;

definition
  let M be type extends commutative Magma;

```

Proposed Evo sketch:

```

  theorem PermProduct[M]:
    ...
end;

```

Proposed Evo sketch:

```
definition
  let T be type;
  struct segment() where
    dist carrier -> T;
    dist limsup -> kindy of T;
  end;
end;

definition
  let S be type extends commutativeagma;
end;
```

Proposed Evo sketch:

```
theorem FermatProduct IM1 :
  ...
end;
```

Presenter script: Say:

- Source design vocabulary: `doc/spec/en/18.templates.md` defines templates as generics for parameterized definitions and theorem schemas, including structures, attributes, modes, predicates, functors, and schemes.

Message:

- Templates are the generics mechanism for Mizar Evo.
- They parameterize definitions and theorem schemas over types, predicates, or functors.
- Classical scheme-style reasoning becomes part of one template vocabulary rather than a separate special case.
- `of` / `over` forms remain readable shorthands; the bracket form records the canonical parameterization.

Design Reason:

- Templates avoid copying the same algebraic construction for every carrier, field, or predicate instance.
- They make generic mathematical patterns explicit enough for dependency fingerprints, theorem indexes, and AI retrieval.
- Constraints such as `type extends commutative Magma` say what the parameter must provide before ATP or proof search begins.

Message:

- Templates are the generics mechanism for Mizar-Evo.
- They parameterize definitions and theorem schemas over types, predicates, or functors.
- Classical scheme-style reasoning becomes part of one template vocabulary rather than a separate special case.
- of / over forms remain readable shorthands; the bracket form records the canonical parameterization.

Design Reason:

- Templates avoid copying the same algebraic construction for every carrier, field, or predicate instance.
- They reuse generic mathematical patterns explicit enough for dependency tracking, theorem indexes, and all other tool.
- Constraints such as type extendable commutative Magma say what the parameter must provide before ATP or proof search begins.

Presenter script: Read aloud:

- Templates are the generics mechanism for Mizar Evo.
- They parameterize definitions and theorem schemas over types, predicates, or functors.
- Classical scheme-style reasoning becomes part of one template vocabulary rather than a separate special case.
- of / over forms remain readable shorthands; the bracket form records the canonical parameterization.
- Templates avoid copying the same algebraic construction for every carrier, field, or predicate instance.

Potential Evo sketch:

```
definition
  let x, y be Integer;

  algorithm max2(x, y) -> Integer
    ensures (result = x or result = y) & x <= result & y <= result
  do
    if x <= y do
      return y;
    end;
    return x;
```

Potential Evo sketch:

```
end;
end;
```

Potential Evo search:

```
definition
  let x, y be Integer;
  registration macc (x, y) => Integer
  ensure :: result = x or result = y if x <= result & y <= result
  do
    if x <= y do
      result y;
    else
      result x;
    end
  end
```

Potential Evo search:

```
end;
end;
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Algorithm Verification (1/2). Point to the visible example, question, or diagram before moving on.

Message:

- Algorithms broaden the scope beyond pure mathematics.
- Contracts and generated VCs must still feed the same verification pipeline.

Message:

- Algorithms broaden the scope beyond pure mathematics.
- Contracts and generated VCs must still feed the same verification pipeline.

Presenter script: Read aloud:

- Algorithms broaden the scope beyond pure mathematics.
- Contracts and generated VCs must still feed the same verification pipeline.

```
@[origin("mizar:ALGSTR_0:addMagma")]  
definition  
  struct AddMagma where  
    ...  
  end;  
end;
```

Message:

- Migration must preserve historical identity.
- Origin metadata supports Formalized Mathematics links, citation stability, and review.

```
0 is origin() "mizar:ALGEBRA_1:addition" ()
def in the
  a term: Addition where
    ...
  end;
end;
```

Message:

- Migration must preserve historical identity.
- Origin metadata supports Formalized Mathematics links, citation stability, and review.

Presenter script: Read aloud:

- Migration must preserve historical identity.
- Origin metadata supports Formalized Mathematics links, citation stability, and review.

After the example, say which design obligation the syntax makes visible.

Questions:

- Which legacy constructs require direct syntax preservation?
- Which constructs should be translated into clearer Evo forms?
- What origin identifiers are stable enough for MML migration?
- Which current environment dependencies are most difficult to explain?

Questions:

- Which legacy constructs require direct syntax preservation?
- Which constructs should be translated into clearer Evo forms?
- What origin identifiers are stable enough for MML migration?
- Which current environment dependencies are most difficult to explain?

Presenter script: Read aloud:

- Which legacy constructs require direct syntax preservation?
- Which constructs should be translated into clearer Evo forms?
- What origin identifiers are stable enough for MML migration?
- Which current environment dependencies are most difficult to explain?

Source

```
-> TokenStream
-> SurfaceAst
-> ResolvedAst
-> TypedAst
-> CoreIr
-> VcIr
-> AtpProblem
-> ProofCertificate
-> VerifiedArtifact
```

Message:

- Each layer separates a different responsibility.
- The separation is for diagnostics, caching, trust, and reproducibility.
- The point is not a generic compiler diagram. Each boundary says who owns a fact, which artifact records it, and what must be recomputed when it changes.

```

graph TD
    subgraph "3.1 - Pipeline Overview"
        direction TB
        L1[Layer 1] --> L2[Layer 2]
        L2 --> L3[Layer 3]
        L3 --> L4[Layer 4]
        L4 --> L5[Layer 5]
        L5 --> L6[Layer 6]
        L6 --> L7[Layer 7]
        L7 --> L8[Layer 8]
        L8 --> L9[Layer 9]
        L9 --> L10[Layer 10]
    end

```

Message:

- Each layer separates a different responsibility.
- The separation is for diagnostics, caching, trust, and reproducibility.
- The point is not a generic compiler diagram. Each boundary says who owns a fact, which artifact records it, and what must be recomputed when it changes.

Presenter script: Read aloud:

- Each layer separates a different responsibility.
- The separation is for diagnostics, caching, trust, and reproducibility.
- The point is not a generic compiler diagram. Each boundary says who owns a fact, which artifact records it, and what must be recomputed when it changes.

After the example, say which design obligation the syntax makes visible.

3.2 - Responsibility Split (1/2)

Layer	Responsibility
frontend	source, lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP	search for evidence
kernel	acceptance by replay/checking

Label	Design Reason
Control	control, manage, parsing, recovery
Problem	problem, error, bugs, non-specs
Domain	data, domain, requirements, models
Algorithm	algorithm, computation
UI/Generate	print and display algorithms
API	search for evidence
Kernel	operation on the representation

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Responsibility Split (1/2). Point to the visible example, question, or diagram before moving on.

3.2 - Responsibility Split (2/2)

Layer	Responsibility
artifact emitter	stable outputs for tools and dependents

Layer	Design rationale
UI/UX: minimal	UI/UX: maps to list to show dependencies

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Responsibility Split (2/2). Point to the visible example, question, or diagram before moving on.

Mizar-side semantics

-> well-typed, resolved obligations

ATP-side search

-> candidate proof evidence

kernel-side checking

-> accepted or rejected proof status

Key point:

- ATPs do not resolve names, infer types, expand clusters, or decide overloads.
- The kernel does not search for proofs.
- Deterministic pre-ATP discharge also needs replayable evidence or an explicit policy status; it is not accepted just because an earlier phase said "done".

```

Mizar-ATP side responsibilities
-> will attempt, resolved obligations
ATP-ATP side search
-> candidate proof evidence
kernel-ATP side checking
-> accepted or rejected proof status

```

Key point:

- ATPs do not resolve names, infer types, expand clusters, or decide overloads.
- The kernel does not search for proofs.
- Deterministic pre-ATP discharge also needs replayable evidence or an explicit policy status; it is not accepted just because an earlier phase said "done".

Presenter script: Read aloud:

- ATPs do not resolve names, infer types, expand clusters, or decide overloads.
- The kernel does not search for proofs.
- Deterministic pre-ATP discharge also needs replayable evidence or an explicit policy status; it is not accepted just because an earlier phase said "done".

After the example, say which design obligation the syntax makes visible.

Design Reason:

- Putting all reasoning inside Mizar would make the trusted verifier larger.
- Delegating all reasoning to ATPs would lose control of Mizar-specific semantics.
- The hybrid boundary keeps semantic processing deterministic, ATP search powerful, and final acceptance independently checkable.

Status distinction:

Status	Meaning
kernel_verified	replayed or certificate-checked evidence accepted
discharged_builtin	deterministic discharge accepted by the same evidence discipline
externally_attested	recorded backend or policy attestation, not equivalent to verified
open / policy status	unfinished or explicitly policy-controlled, not silently promoted

Design Reason:

- Putting all reasoning inside Mizar would make the trusted verifier larger.
- Delegating all reasoning to ATPs would lose control of Mizar-specific semantics.
- The hybrid boundary keeps semantic processing deterministic, ATP search powerful, and final acceptance independently checkable.

Status distinction:

Proofs	Meaning
normal_discharged	explored or certificate-checked evidence accepted
discharged_validation	deterministic discharges accepted by the same evidence checker
acceptance_certificate	external evidence as proof of semantic, not syntactic, validity
open / proof status	not related to explicit proof-control, not directly proved

Presenter script: Read aloud:

- Putting all reasoning inside Mizar would make the trusted verifier larger.
- Delegating all reasoning to ATPs would lose control of Mizar-specific semantics.
- The hybrid boundary keeps semantic processing deterministic, ATP search powerful, and final acceptance independently checkable.

For the table, read the row labels first, then the design reason.

Explanation:

- High-level proof search may use ATPs.
- Accepted evidence is normalized into certificate data.
- The kernel checks imported facts, substitutions, alpha-conversion, clause well-formedness, and resolution/SAT evidence.
- Unsoundness in search should not become unsoundness in accepted results.
- The SAT part is proof evidence checking, not trust in a solver's success bit. Backend-specific proofs must be translated into replayable certificate data before acceptance.

Explanation:

- High-level proof search may use ATPs.
- Accepted evidence is normalized into certificate data.
- The kernel checks imported facts, substitutions, alpha-conversion, clause well-formedness, and resolution/SAT evidence.
- Unsoundness in search should not become unsoundness in accepted results.
- The SAT part is proof evidence checking, not trust in a solver's success bit. Backend-specific proofs must be translated into replayable certificate data before acceptance.

Presenter script: Read aloud:

- High-level proof search may use ATPs.
- Accepted evidence is normalized into certificate data.
- The kernel checks imported facts, substitutions, alpha-conversion, clause well-formedness, and resolution/SAT evidence.
- Unsoundness in search should not become unsoundness in accepted results.
- The SAT part is proof evidence checking, not trust in a solver's success bit. Backend-specific proofs must be translated into replayable certificate data before acceptance.

Certificate

- target VC fingerprint
- kernel profile
- imported facts and hashes
- generated clauses
- substitutions
- resolution trace
- final goal

Message:

- The certificate binds a proof to a specific obligation and dependency slice.
- It must be deterministic and replayable.
- This is why the certificate carries hashes and a kernel profile: a proof accepted under one dependency and policy context should not silently migrate to another.

```

Certificate
  target: string
  kernel: profile
  imported: factor and bundle
  generated: clause
  signature: clause
  proof: and clause
  goal: goal

```

Message:

- The certificate binds a proof to a specific obligation and dependency slice.
- It must be deterministic and replayable.
- This is why the certificate carries hashes and a kernel profile: a proof accepted under one dependency and policy context should not silently migrate to another.

Presenter script: Read aloud:

- The certificate binds a proof to a specific obligation and dependency slice.
- It must be deterministic and replayable.
- This is why the certificate carries hashes and a kernel profile: a proof accepted under one dependency and policy context should not silently migrate to another.

After the example, say which design obligation the syntax makes visible.

Key Points:

- no proof search;
- no premise selection;
- no name resolution;
- no overload resolution;
- no cluster expansion;
- no hidden global state;
- no trust in raw ATP success.

Key Points:

- no proof search;
- no premise selection;
- no name resolution;
- no overload resolution;
- no cluster expansion;
- no hidden global state;
- no trust in raw ATP success.

Presenter script: Read aloud:

- no proof search;
- no premise selection;
- no name resolution;
- no overload resolution;
- no cluster expansion;

Message:

- Mizar's registrations are a strength but need traceability at scale.
- Evo stores registration and cluster data in indexes.
- An active module gets an import-filtered view.
- Explanation artifacts expose why a fact was or was not derived.

Message:

- Mizar's registrations are a strength but need traceability at scale.
- Evo stores registration and cluster data in indexes.
- An active module gets an import-filtered view.
- Explanation artifacts expose why a fact was or was not derived.

Presenter script: Read aloud:

- Mizar's registrations are a strength but need traceability at scale.
- Evo stores registration and cluster data in indexes.
- An active module gets an import-filtered view.
- Explanation artifacts expose why a fact was or was not derived.

3.8 - Memory Model

resident memory should scale with:

- active source
- imported public interfaces
- import-filtered indexes
- active module VCs

not with:

- imported proof bodies
- private lemmas outside the interface
- unused registration data outside the import closure

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Memory Model. Point to the visible example, question, or diagram before moving on.

Main points:

- Hash source, dependency interfaces, generated VCs, and proof witnesses.
- Reuse only when fingerprints and verifier policy match.
- Proof-body-only changes should not rebuild unrelated importers when public statements and statuses remain unchanged.

- Hash source, dependency interfaces, generated VCs, and proof witnesses.
- Reuse only when fingerprints and verifier policy match.
- Proof-body-only changes should not rebuild unrelated importers when public statements and statuses remain unchanged.

Presenter script: Read aloud:

- Hash source, dependency interfaces, generated VCs, and proof witnesses.
- Reuse only when fingerprints and verifier policy match.
- Proof-body-only changes should not rebuild unrelated importers when public statements and statuses remain unchanged.

Main points:

- Parallelize independent modules, obligations, ATP runs, and kernel checks.
- Publish diagnostics, artifacts, and proof statuses in canonical order.
- Runtime completion order must not determine semantics.

- Parallelize independent modules, obligations, ATP runs, and kernel checks.
- Publish diagnostics, artifacts, and proof statuses in canonical order.
- Runtime completion order must not determine semantics.

Presenter script: Read aloud:

- Parallelize independent modules, obligations, ATP runs, and kernel checks.
- Publish diagnostics, artifacts, and proof statuses in canonical order.
- Runtime completion order must not determine semantics.

Artifacts should serve:

- downstream package verification;
- IDE hover, diagnostics, and go-to-definition;
- AI context extraction and patch verification;
- documentation generation;
- Formalized Mathematics article links;
- reproducible build records.

- downstream package verification;
- IDE hover, diagnostics, and go-to-definition;
- AI context extraction and patch verification;
- documentation generation;
- Formalized Mathematics article links;
- reproducible build records.

Presenter script: Read aloud:

- downstream package verification;
- IDE hover, diagnostics, and go-to-definition;
- AI context extraction and patch verification;
- documentation generation;
- Formalized Mathematics article links;

Message:

- LSP gives editor feedback from syntax and semantic artifacts.
- Planned MCP-style resources and tools expose bounded verifier context to AI agents.
- Both should consume artifacts rather than reimplementing the verifier.
- The wire protocol is not the design center here; the design center is that agents read bounded, typed, auditable context and cannot bypass verification.
- The current fixed policy is the Green/Yellow/Red edit classification and authorization scopes; detailed wire schemas remain roadmap material.

Message:

- LSP gives editor feedback from syntax and semantic artifacts.
- Planned MCP-style resources and tools expose bounded verifier context to AI agents.
- Both should consume artifacts rather than reimplementing the verifier.
- The wire protocol is not the design center here; the design center is that agents read bounded, typed, auditable context and cannot bypass verification.
- The current fixed policy is the Green/Yellow/Red edit classification and authorization scopes; detailed wire schemas remain roadmap material.

Presenter script: Read aloud:

- LSP gives editor feedback from syntax and semantic artifacts.
- Planned MCP-style resources and tools expose bounded verifier context to AI agents.
- Both should consume artifacts rather than reimplementing the verifier.
- The wire protocol is not the design center here; the design center is that agents read bounded, typed, auditable context and cannot bypass verification.
- The current fixed policy is the Green/Yellow/Red edit classification and authorization scopes; detailed wire schemas remain roadmap material.

3.13 - Safe AI Edit Classes

Class	Examples	Policy
Green	add citation, insert qua, add info annotation	may be auto-proposed, still verified
Yellow	add import, local lemma, registration, invariant	proposed with human review
Red	add axiom, weaken theorem, change definition, relax kernel	forbidden to ordinary AI agents

Class	Example	Notes
Class	add classes, modify, add the associated	may be auto-generated, or added
When	add to pool, local form, or register with, instance	stop and with human review
Do	add items, remove items, change metadata, rules added	allowed to start with AI agents

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Safe AI Edit Classes. Point to the visible example, question, or diagram before moving on.

Main principle:

```
Reject what must not pass  
before  
Accept everything that should pass
```

Talk angle:

- Parser gaps are annoying.
- Soundness bugs are dangerous.
- Kernel-adjacent tests should emphasize malformed and failing inputs.
- The test mix should therefore be intentionally asymmetric near the trust boundary: reject invalid evidence first, then broaden accepted-language coverage.

Main principle:

Reject what must not pass
before
Accept everything that should pass

Talk angle:

- Parser gaps are annoying.
- Soundness bugs are dangerous.
- Kernel-adjacent tests should emphasize malformed and failing inputs.
- The test mix should therefore be intentionally asymmetric near the test boundary: reject invalid evidence first, then broader accepted-but-dangerous cases.

Presenter script: Read aloud:

- Parser gaps are annoying.
- Soundness bugs are dangerous.
- Kernel-adjacent tests should emphasize malformed and failing inputs.

After the example, say which design obligation the syntax makes visible.

Questions:

- Is the SAT certificate story convincing for Mizar-style proofs?
- Which proof evidence format would be easiest for the Mizar team to audit?
- Which artifact should be built first for real migration experiments?

Questions:

- Is the SAT certificate story convincing for Mizar-style proofs?
- Which proof evidence format would be easiest for the Mizar team to audit?
- Which artifact should be built first for real migration experiments?

Presenter script: Read aloud:

- Is the SAT certificate story convincing for Mizar-style proofs?
- Which proof evidence format would be easiest for the Mizar team to audit?
- Which artifact should be built first for real migration experiments?

Key Phrase:

MML migration is a research program, not a one-shot translation.

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Roadmap Thesis. Point to the visible example, question, or diagram before moving on.

Outputs from the visit:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on language changes;
- a first paper outline;
- a shared list of objections and risks.

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on language changes;
- a first paper outline;
- a shared list of objections and risks.

Presenter script: Read aloud:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on language changes;
- a first paper outline;
- a shared list of objections and risks.

Possible alpha scope:

- frontend and parser for a meaningful core subset;
- import prelude and module resolution prototype;
- structured AST and diagnostics;
- basic type and registration scaffolding;
- test harness and corpus layout;
- early artifact format for parsed/resolved source.

Non-goals:

- full MML verification;
- final compatibility layer;
- stable AI protocol;
- complete Formalized Mathematics workflow.

Possible alpha scope:

- frontend and parser for a meaningful core subset;
- import prelude and module resolution prototype;
- structured AST and diagnostics;
- basic type and registration scaffolding;
- test harness and corpus layout;
- ready artifact format for parsing/resolved sources.

Non-goals:

- full MML verification;
- final compatibility layer;
- attack AI protocol;
- complete Formalized Mathematics unification.

Presenter script: Read aloud:

- frontend and parser for a meaningful core subset;
- import prelude and module resolution prototype;
- structured AST and diagnostics;
- basic type and registration scaffolding;
- test harness and corpus layout;

Focus:

- choose 3-5 representative MML articles;
- translate by hand and with scripts;
- record every mismatch as a classified issue;
- build side-by-side reports.

Suggested issue classes:

- syntax translation;
- environment-to-import mapping;
- structure inheritance;
- registration or cluster behavior;
- overload behavior;
- proof citation or ATP behavior;
- documentation and origin metadata.

Focus:

- choose 3-5 representative MML articles;
- translate by hand and with scripts;
- record every mismatch as a classified issue;
- build side-by-side reports.

Suggested issue classes:

- syntax translation;
- environment-to-impot. mapping;
- abstract vs. concrete;
- regularities or clusters behavior;
- new kind behavior;
- proof derivation vs ATP behavior;
- documentation of origin metadata.

Presenter script: Read aloud:

- choose 3-5 representative MML articles;
- translate by hand and with scripts;
- record every mismatch as a classified issue;
- build side-by-side reports.
- syntax translation;

Candidate path:

- ① foundational set and relation fragments;
- ② functions and binary operations;
- ③ algebraic structures;
- ④ selected theorem-heavy articles;
- ⑤ broader dependency cones around successful fragments.

Candidate path:

- foundational set and relation fragments;
- functions and binary operations;
- algebraic structures;
- selected theorem-heavy articles;
- broader dependency cones around successful fragments.

Presenter script: Read aloud:

- foundational set and relation fragments;
- functions and binary operations;
- algebraic structures;
- selected theorem-heavy articles;
- broader dependency cones around successful fragments.

Measure:

- translated LOC and article count;
- accepted parser subset;
- resolved imports versus unresolved dependencies;
- proof obligations generated;
- obligations closed by deterministic reasoning;
- obligations closed by ATP plus kernel certificate;
- memory and build time per module;

Measure:

- number of compatibility decisions needing human review.

Measure:

- translated LOC and article count;
- accepted parser subset;
- resolved imports versus unresolved dependencies;
- proof obligations generated;
- obligations closed by deterministic reasoning;
- obligations closed by ATP plus derived theorems;
- memory and build time per module;

Measure:

- number of compatibility decisions needing human review.

Presenter script: Read aloud:

- translated LOC and article count;
- accepted parser subset;
- resolved imports versus unresolved dependencies;
- proof obligations generated;
- obligations closed by deterministic reasoning;

Possible policy:

- preserve theorem identity through origin metadata;
- keep source-visible compatibility aliases where they help migration;
- avoid preserving legacy forms that block clear module and artifact semantics;
- record every divergence from old behavior with a reason and test.
- Compatibility should be argued from mathematical identity, review value, and reproducibility, not from nostalgia for every surface form.

Possible policy:

- preserve theorem identity through origin metadata;
- keep source-visible compatibility aliases where they help migration;
- avoid preserving legacy forms that block clear module and artifact semantics;
- record every divergence from old behavior with a reason and test.
- Compatibility should be argued from mathematical identity, review value, and reproducibility, not from nostalgia for every surface form.

Presenter script: Read aloud:

- preserve theorem identity through origin metadata;
- keep source-visible compatibility aliases where they help migration;
- avoid preserving legacy forms that block clear module and artifact semantics;
- record every divergence from old behavior with a reason and test.
- Compatibility should be argued from mathematical identity, review value, and reproducibility, not from nostalgia for every surface form.

4.8 - Migration Risk Table

Risk	Mitigation
syntax compatibility consumes the project	start with representative slices, not full automation
registrations behave differently	build trace and explanation artifacts early
proof search differs from current verifier	record used facts and compare proof obligations
package layout breaks Formalized Mathematics links	origin metadata and article-to-library identifiers
AI edits hide migration mistakes	keep Red edits forbidden and require verifier artifacts

└ Part 4. Roadmap

└ 4.8 - Migration Risk Table

4.8 - Migration Risk Table

Item	Mitigation
Source compatibility: cannot be proven	Start with representative data, not full migration
Implementation differences	Full team and implementer attacks only
Proofs must differ from source code	Proof and source code are proof of migration
Package format: from source to binary	Binary migration and package format
All data migration is binary	Binary migration and package format

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Migration Risk Table. Point to the visible example, question, or diagram before moving on.

Questions:

- Which MML articles are small but structurally representative?
- Which article families stress registrations and structures?
- Which current Mizar idioms are culturally important, not just technically convenient?
- What migration result would convince the community that Evo is serious?

Questions:

- Which MML articles are small but structurally representative?
- Which article families stress registrations and structures?
- Which current Mizar idioms are culturally important, not just technically convenient?
- What migration result would convince the community that Evo is serious?

Presenter script: Read aloud:

- Which MML articles are small but structurally representative?
- Which article families stress registrations and structures?
- Which current Mizar idioms are culturally important, not just technically convenient?
- What migration result would convince the community that Evo is serious?

Key Phrase:

The verified library and the published article should be linked but not forced to have the same structure.

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Publication Thesis. Point to the visible example, question, or diagram before moving on.

Library modules optimize:

- reuse;
- dependency control;
- package versioning;
- stable machine-readable artifacts.

Formalized Mathematics articles optimize:

- exposition;
- narrative;
- citation;
- peer review;
- reader-facing mathematical structure.

Library modules optimize:

- reuse;
- dependency control;
- package versioning;
- stable machine-readable artifacts.

Formalized Mathematics articles optimize:

- exposition;
- notation;
- citation;
- peer review;
- reading/finding mathematical literature.

Presenter script: Read aloud:

- reuse;
- dependency control;
- package versioning;
- stable machine-readable artifacts.
- exposition;

Formalized Mathematics theorem

- > library package
- > module path
- > theorem FQN
- > statement fingerprint
- > verified artifact hash
- > rendered documentation page

```
Formalized Mathematics theorems  
-> library package  
-> module path  
-> theorems TTH  
-> statement fingerprint  
-> verified statement hash  
-> rendered documentation page
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Link Model. Point to the visible example, question, or diagram before moving on.

5.4 - Example Link Record (1/2)

```
article: Formalized Mathematics, future volume
section: Unique identity element
library_object: std.algebra.group.identity_unique
package: std_algebra
version: 0.4.0
statement_hash: sha256:...
artifact_hash: sha256:...
origin: mizar:GROUP_1:...
```


Part 5. Formalized Mathematics

5.4 - Example Link Record (1/2)

```

set column: formula and mathematics, state a column
set column: trigonometry identity theorem
library subject: std.algebra.group.identity.uniqueness
package: std.algebra
version: 0.0.0
set column: value: 0.000000...
set column: value: 0.000000...
set column: value: 0.000000...

```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Example Link Record (1/2). Point to the visible example, question, or diagram before moving on.

Design Reason:

- These identities are deliberately layered.
- The article label supports scholarly citation and narrative structure.
- The origin id preserves continuity with MML and existing Formalized Mathematics history.
- The library FQN addresses the current reusable theorem object.
- The statement fingerprint detects semantic drift.
- The artifact hash supports reproducible verification of the exact build.

Design Reason:

- These identities are deliberately layered.
- The article label supports scholarly citation and narrative structure.
- The origin id preserves continuity with MML and existing Formalized Mathematics history.
- The library FQN addresses the current reusable theorem object.
- The statement fingerprint detects semantic drift.
- The effect hash supports reproducible verification of the exact build.

Presenter script: Read aloud:

- These identities are deliberately layered.
- The article label supports scholarly citation and narrative structure.
- The origin id preserves continuity with MML and existing Formalized Mathematics history.
- The library FQN addresses the current reusable theorem object.
- The statement fingerprint detects semantic drift.

For readers:

- article prose remains primary;
- formal source is one click away;
- proof status and dependency information are inspectable.

For maintainers:

- library refactoring does not automatically rewrite article exposition;
- versioned links preserve reproducibility;
- origin metadata preserves historical continuity.

For AI:

- article prose provides semantic search text;
- formal links provide exact theorem identities;
- agents can retrieve both explanation and machine-checkable context.

For readers:

- article prose remains primary;
- formal source is one click away;
- proof status and dependency information are inspectable.

For maintainers:

- library refactoring does not automatically rewrite article exposition;
- versioned links preserve reproducibility;
- digital metadata preserves historical context.

For AI:

- article prose provides semantic search text;
- formal links provide exact theorem identifiers;
- agents can retrieve both explanation and machine-checkable context.

Presenter script: Read aloud:

- article prose remains primary;
- formal source is one click away;
- proof status and dependency information are inspectable.
- library refactoring does not automatically rewrite article exposition;
- versioned links preserve reproducibility;

Questions:

- Should stable identity be based on library FQN, article label, origin id, or artifact hash?
- Which of these identities should be primary in user-facing citations, and which should remain machine-facing reproducibility checks?
- How should article revisions track library refactoring?
- Should Formalized Mathematics publish generated HTML, PDF, or both?
- How should old Formalized Mathematics articles link to migrated library modules?

Questions:

- Should stable identity be based on library FQN, article label, origin id, or artifact hash?
- Which of these identities should be primary in user-facing citations, and which should remain machine-facing reproducibility checks?
- How should article revisions track library refactoring?
- Should Formalized Mathematics publish generated HTML, PDF, or both?
- How should old Formalized Mathematics articles link to migrated library modules?

Presenter script: Read aloud:

- Should stable identity be based on library FQN, article label, origin id, or artifact hash?
- Which of these identities should be primary in user-facing citations, and which should remain machine-facing reproducibility checks?
- How should article revisions track library refactoring?
- Should Formalized Mathematics publish generated HTML, PDF, or both?
- How should old Formalized Mathematics articles link to migrated library modules?

```
library theorem verifies
```

- > verified artifact emitted
- > documentation generated
- > article references formal objects
- > publication build checks links and versions
- > article published with formal links


```
library has been verified  
-> verified set of set emitted  
-> document is generated  
-> article references formal objects  
-> publication build the book and version  
-> article published with formal links
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Possible Workflow. Point to the visible example, question, or diagram before moving on.

Key Points:

- language identity and compatibility;
- MML migration examples;
- registration and cluster semantics;
- proof evidence and kernel boundary;
- Formalized Mathematics link model;
- roadmap realism.

Key Points:

- language identity and compatibility;
- MML migration examples;
- registration and cluster semantics;
- proof evidence and kernel boundary;
- Formalized Mathematics link model;
- reading system.

Presenter script: Read aloud:

- language identity and compatibility;
- MML migration examples;
- registration and cluster semantics;
- proof evidence and kernel boundary;
- Formalized Mathematics link model;

Proposed next steps:

- ➊ Select migration benchmark articles.
- ➋ Prepare exact old/new code comparison slides.
- ➌ Write an initial MML migration report template.
- ➍ Refine this detailed Beamer deck after Bialystok review.
- ➎ Start the paper outline using the same main claims and review feedback.

- Select migration benchmark articles.
- Prepare exact old/new code comparison slides.
- Write an initial MML migration report template.
- Refine this detailed Beamer deck after Bialystok review.
- Start the paper outline using the same main claims and review feedback.

Presenter script: Read aloud:

- Select migration benchmark articles.
- Prepare exact old/new code comparison slides.
- Write an initial MML migration report template.
- Refine this detailed Beamer deck after Bialystok review.
- Start the paper outline using the same main claims and review feedback.

Final slide:

Mizar Evo should be modern where scale demands it,
and conservative where Mizar's mathematical identity depends on it.

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Closing. Point to the visible example, question, or diagram before moving on.

Prepared short excerpts for Beamer conversion:

Article environment Source: algstr_0.miz

Lines: 15-25

Migration point: Translate article-level dependency classes into explicit import/package roles.

Evo sketch: `import std.function.binop; import std.algebra.structure.sorted;`

Structure definition Source: algstr_0.miz

Lines: 37-40

Migration point: Preserve structures while separating intrinsic fields from canonical properties and inheritance obligations.

Evo sketch: `field carrier, field add, and, where appropriate, property zero / property unit`

Registration/cluster propagation Source: algstr_0.miz

Lines: 104-109

Migration point: Keep registrations, but make propagation traceable and import-filtered.

Evo sketch: registration graph entry plus resolution trace artifact

└ Backup A. Prepared Exact Examples

└ Backup A. Prepared Exact Examples (1/3)

```

\hidebackmatter Source: mizar_1.miz
Line: 1528
Mizarithm path: Translating mizarlib's theorems to first order logic in path/package miz.
Low abstric: none. Use abstract theory: none. Use abstract theory: none.
Structure definition Source: mizar_1.miz
Line: 1530
Mizarithm path: Translating mizarlib's theorems to first order logic in path/package miz.
Low abstric: none. Use abstract theory: none. Use abstract theory: none.
\begin{document} Source: mizar_1.miz
Line: 1531
Mizarithm path: Translating mizarlib's theorems to first order logic in path/package miz.
Low abstric: none. Use abstract theory: none. Use abstract theory: none.
\end{document}

```

Presenter script: For the table, read the row labels first, then the design reason.
 Use this as the transition: Backup A. Prepared Exact Examples (1/3). Point to the visible example, question, or diagram before moving on.

Prepared short excerpts for Beamer conversion:

Proof citation **Source:** `struct_0.miz`

Lines: 637-643

Migration point: Treat citation repair as a small verifier-checked edit.

Evo sketch: Green edit: add or refine by citations; artifact records used facts

Publication link **Source:** `contents.html`

Lines: 516-517

Migration point: Link scholarly article metadata to a reusable MML/library identifier.

Evo sketch: Formalized Mathematics article label + origin: `mizar:GROUP_1 + theorem`
FQN/fingerprint

Source URLs:

- ALGSTR_0: `<https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>`
- STRUCT_0: `<https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>`
- Formalized Mathematics contents: `<https://mizar.uwb.edu.pl/fm/contents.html>`

```

Final status: 200102:10002_0.miz
License: 037443
Mizarlib path: /usr/lib/mizar/mizarlib
License path: /usr/lib/mizar/mizarlib
Published by: 200102:10002_0.miz
License: 037443
Mizarlib path: /usr/lib/mizar/mizarlib
License path: /usr/lib/mizar/mizarlib

```

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- Formalized Mathematics contents: <<https://mizar.uwb.edu.pl/fm/contents.html>>

Presenter script: Read aloud:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- Formalized Mathematics contents: <<https://mizar.uwb.edu.pl/fm/contents.html>>

For the table, read the row labels first, then the design reason.

Remaining Beamer preparation:

- Decide whether the visible slide should show the raw source URL, a shortened citation label, or both.
- Add a final attribution note for MML source licensing.
- If space is tight, move exact source URLs to speaker notes and keep only article names and line numbers on slides.

Remaining Beamer preparation:

- Decide whether the visible slide should show the raw source URL, a shortened citation label, or both.
- Add a final attribution note for MML source licensing.
- If space is tight, move exact source URLs to speaker notes and keep only article names and line numbers on slides.

Presenter script: Read aloud:

- Decide whether the visible slide should show the raw source URL, a shortened citation label, or both.
- Add a final attribution note for MML source licensing.
- If space is tight, move exact source URLs to speaker notes and keep only article names and line numbers on slides.

Required diagrams:

- 1 Current Mizar workflow: source -> Accommodator -> Verifier -> Exporter -> MML.
- 2 Three pillars: readability, AI-readiness, scalability.
- 3 Environment-to-import migration.
- 4 Structure inheritance and diamond coherence.
- 5 Full compiler/verifier pipeline.
- 6 Reasoning boundary: Mizar semantics / ATP search / kernel checking.
- 7 Certificate object and SAT/UNSAT checking.

Required diagrams:

- 1 Incremental dependency and fingerprint graph.
- 2 AI patch verification flow.
- 3 Formalized Mathematics article-to-library link model.
- 4 Roadmap timeline.

Mizar Evo

└ Backup B. Diagram List

└ Backup B. Diagram List

Required diagrams:

- Current Mizar workflow: source -> Accommodator -> Verifier -> Exporter -> MML.
- Three pillars: readability, AI-readiness, scalability.
- Environment-to-import migration.
- Structure inheritance and diamond coherence.
- Full compiler/verifier pipeline.
- Reasoning boundary: Mizar semantics / ATP search / formal checking.
- Certificate object and SAT/UNSAT checking.

Required diagrams:

- Interactive dependency and fingerprint graph.
- AI patch verification flow.
- Formalized Mathematics rich-to-library list model.
- Roadmap timeline.

Presenter script: Read aloud:

- Current Mizar workflow: source -> Accommodator -> Verifier -> Exporter -> MML.
- Three pillars: readability, AI-readiness, scalability.
- Environment-to-import migration.
- Structure inheritance and diamond coherence.
- Full compiler/verifier pipeline.

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- ➊ Introduction: why Mizar needs evolution now.
- ➋ Mizar as baseline: readability, MML, Formalized Mathematics.
- ➌ Design principles.
- ➍ Language evolution and compatibility.
- ➎ Verifier architecture and small kernel.
- ➏ AI-safe proof development.
- ➐ Package-based library and publication workflow.

Mizar Evo

└ Backup C. Paper Outline Seed

└ Backup C. Paper Outline Seed (1/2)

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles.
- Language evolution and compatibility.
- Verifier architecture and small is real.
- Axiom proof development.
- Package-based library of publications workflow.

Presenter script: Read aloud:

- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles.

After the example, say which design obligation the syntax makes visible.

Possible sections:

- ➊ Migration plan and evaluation metrics.
- ➋ Related work: current Mizar, Lean, Isabelle/Isar, ATP-integrated systems.
- ➌ Conclusion and collaboration agenda.

Mizar Evo

└ Backup C. Paper Outline Seed

└ Backup C. Paper Outline Seed (2/2)

Possible sections:

- Migration plan and evaluation metrics.
- Related work: current Mizar, Lean, Isabelle/HOL, ATP-integrated systems.
- Conclusion and collaboration agenda.

Presenter script: Read aloud:

- Migration plan and evaluation metrics.
- Related work: current Mizar, Lean, Isabelle/HOL, ATP-integrated systems.
- Conclusion and collaboration agenda.

Use this checklist before converting to Beamer:

- Does every criticism of current practice also acknowledge why that practice was useful?
- Does every Lean comparison avoid sounding dismissive?
- Does every architecture claim map to an existing or planned artifact?
- Does every code example say whether it is exact, schematic, or proposed?
- Do structure slides explain why `field`, `property`, and `attribute` are distinct?
- Do inheritance slides explain why `struct` and `inherit` are separate declarations?
- Do template slides explain why generic definitions and theorem schemas need a first-class mechanism rather than copy-paste or ad hoc schemes?

Use this checklist before convening to Beamer:

- Does every criticism of current practice also acknowledge why that practice was useful?
- Does every lean comparison avoid sounding dismissive?
- Does every architecture claim map to an existing or planned artifact?
- Does every code example say whether it is exact, schematic, or proposed?
- Do structure slides explain why field, property, and attribute are distinct?
- Do instance slides explain why struct and lambda are separate declarations?
- Do template slides explain why generic definitions are those mechanisms that need a first-class mechanism rather than copy-paste or ad hoc schemes?

Presenter script: Read aloud:

- Does every criticism of current practice also acknowledge why that practice was useful?
- Does every Lean comparison avoid sounding dismissive?
- Does every architecture claim map to an existing or planned artifact?
- Does every code example say whether it is exact, schematic, or proposed?
- Do structure slides explain why field, property, and attribute are distinct?

Use this checklist before converting to Beamer:

- Are Red AI edits clearly forbidden?
- Are MML migration claims measurable?
- Does the Formalized Mathematics section preserve the value of publication, not only library indexing?

- Are Red AI edits clearly forbidden?
- Are MML migration claims measurable?
- Does the Formalized Mathematics section preserve the value of publication, not only library indexing?

Presenter script: Read aloud:

- Are Red AI edits clearly forbidden?
- Are MML migration claims measurable?
- Does the Formalized Mathematics section preserve the value of publication, not only library indexing?